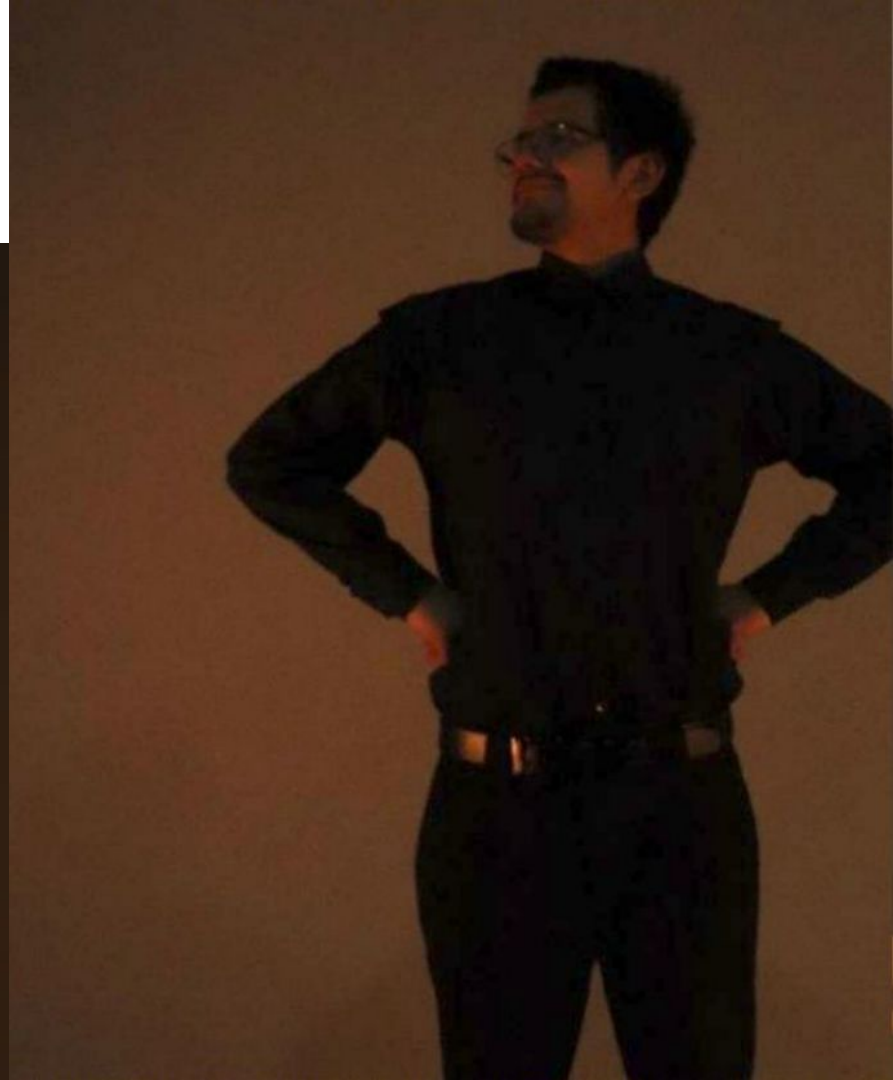
Programación Dinámica

Dedicado al maestro del DP



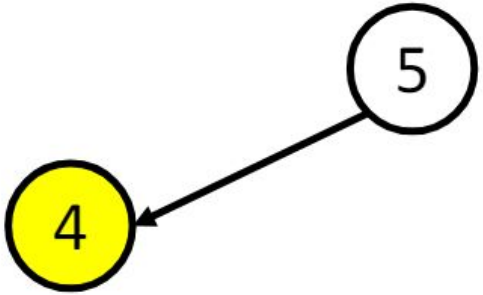
Fibonacci

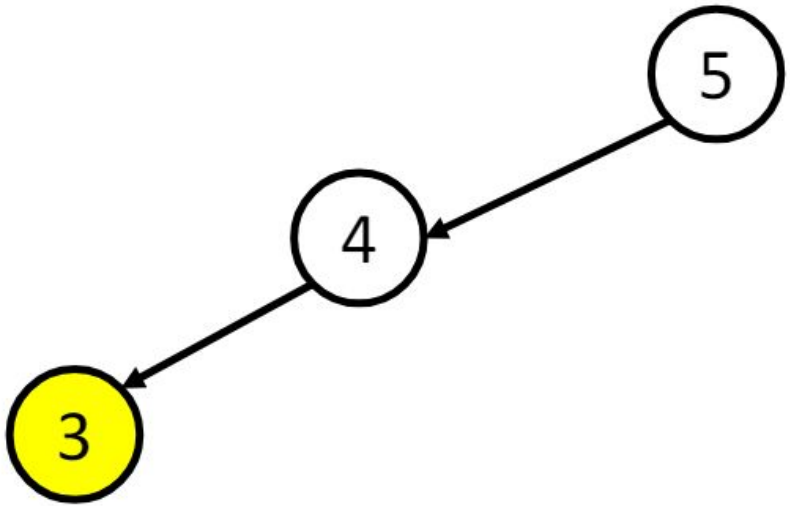
La sucesión de Fibonacci se define como $f_0 = 0$, $f_1 = 1$, $f_n = f_{n-1} + f_{n-2}$ (para $n \geq 2$). Los primeros términos son 0, 1, 1, 2, 3, 5, 8, 13, 21…

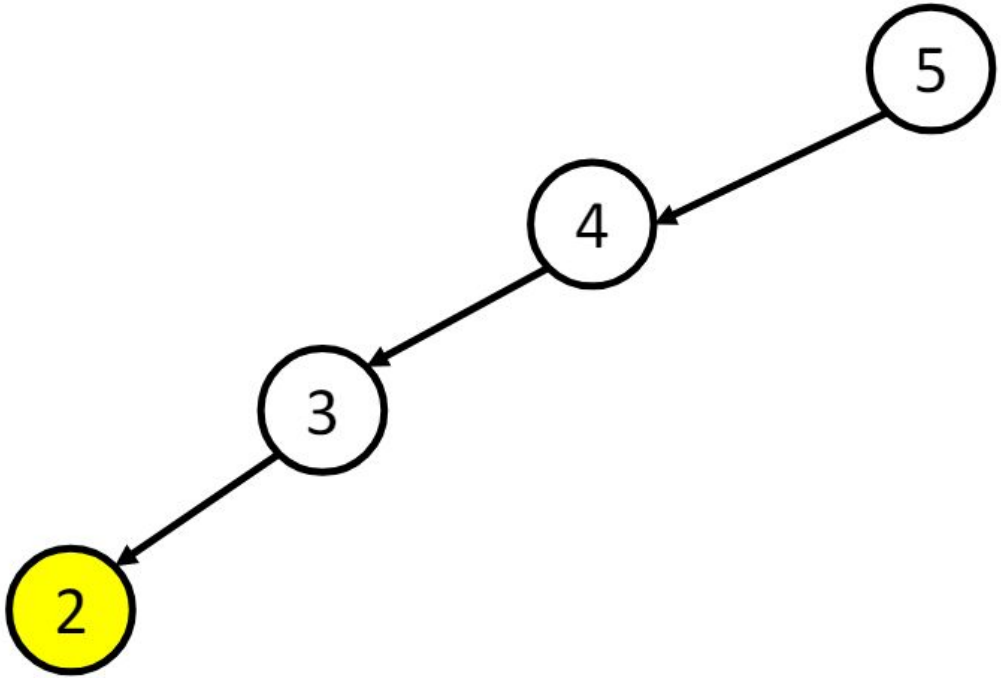
Código recursivo complejidad exponencial:

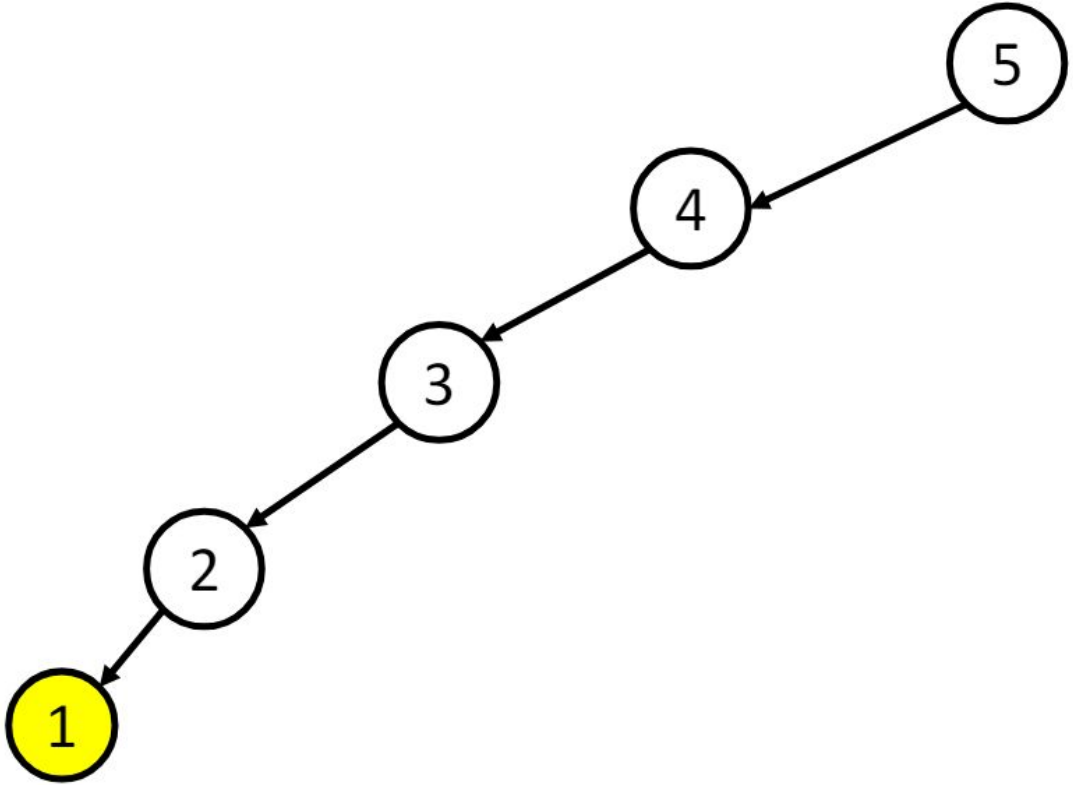
```
int fib(int n) {
    if(n <= 1)
        return n;
    return fib(n - 1) + fib(n - 2);
}
```

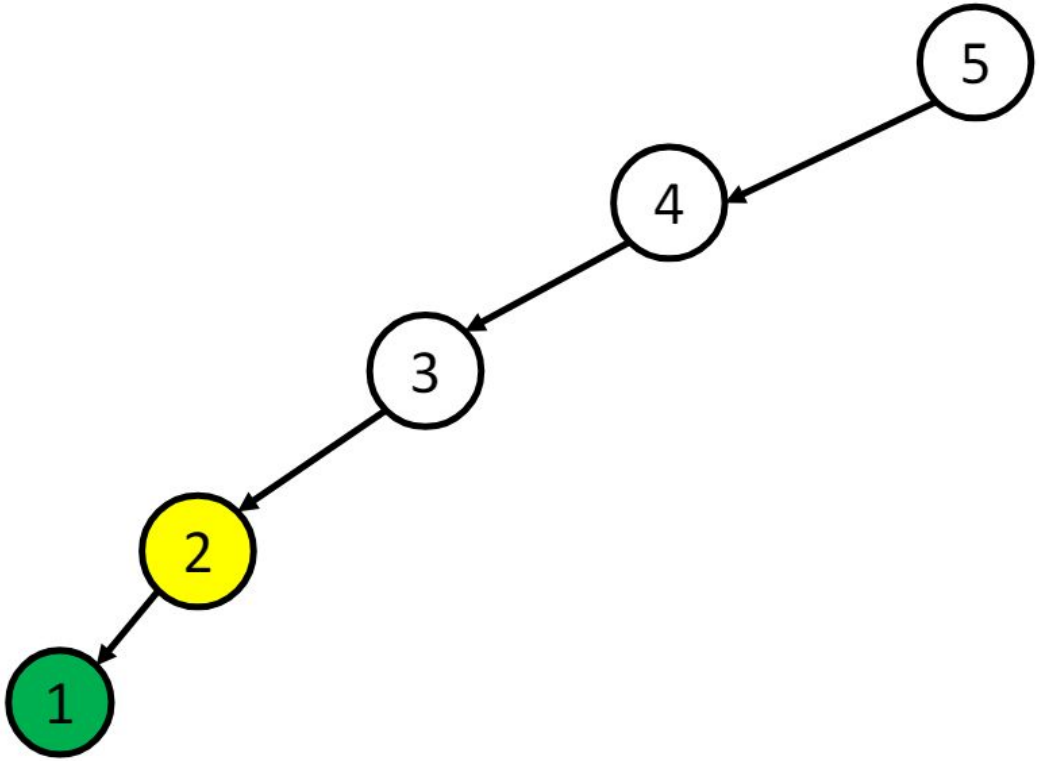
5

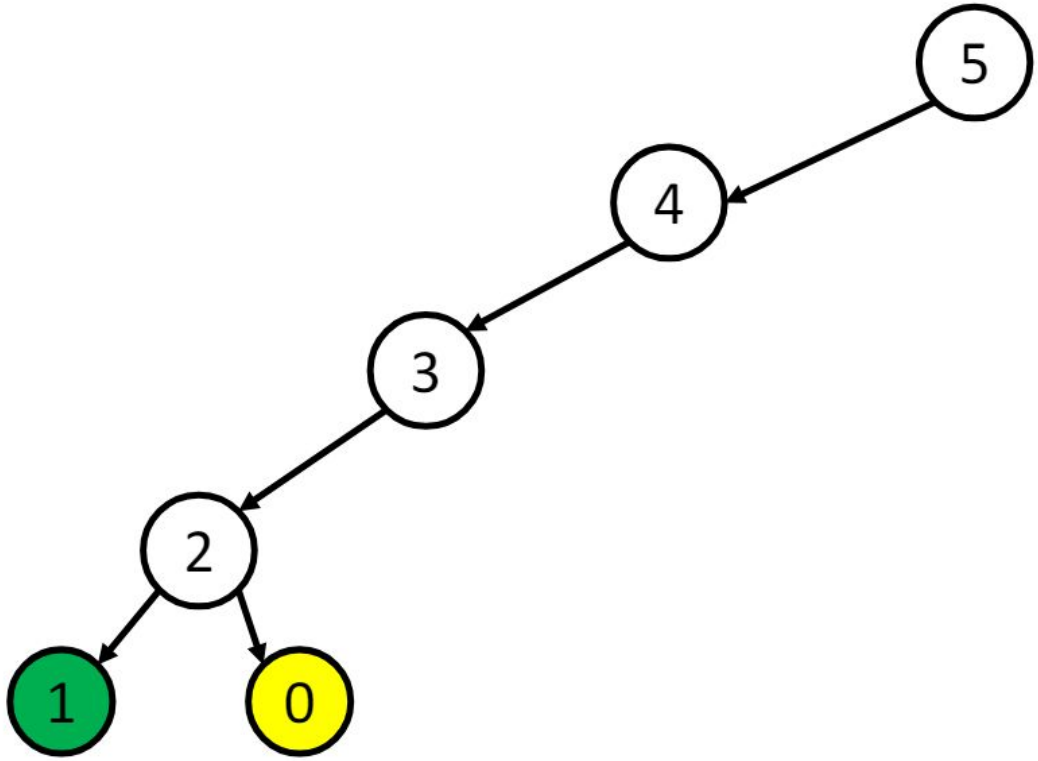

5
4

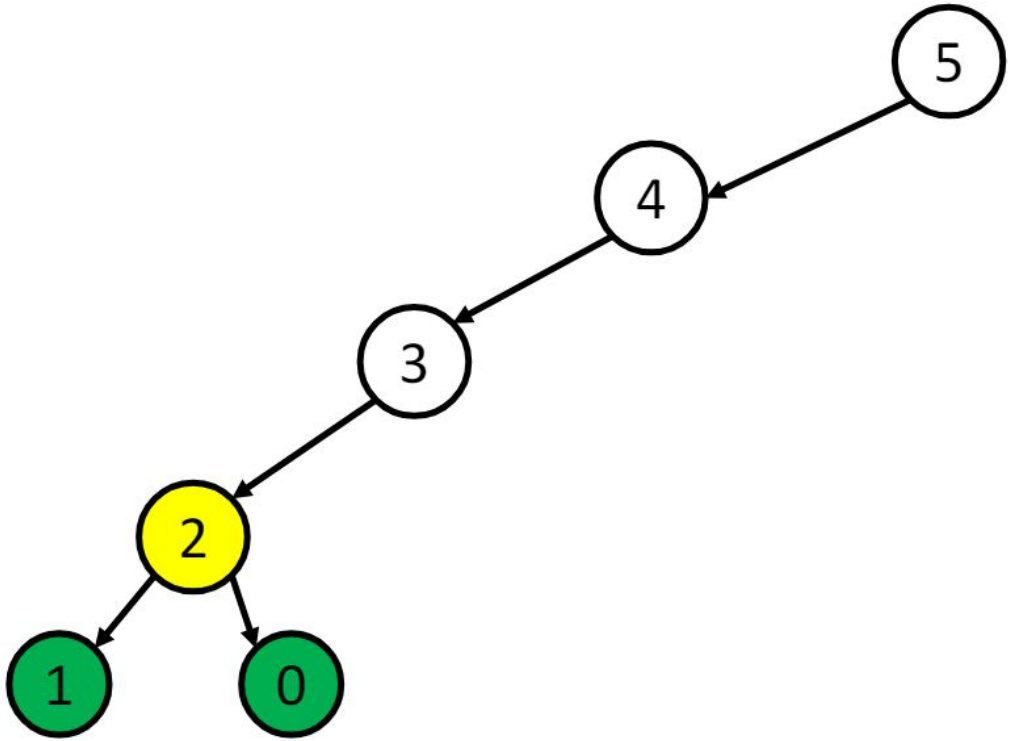

5
4
3

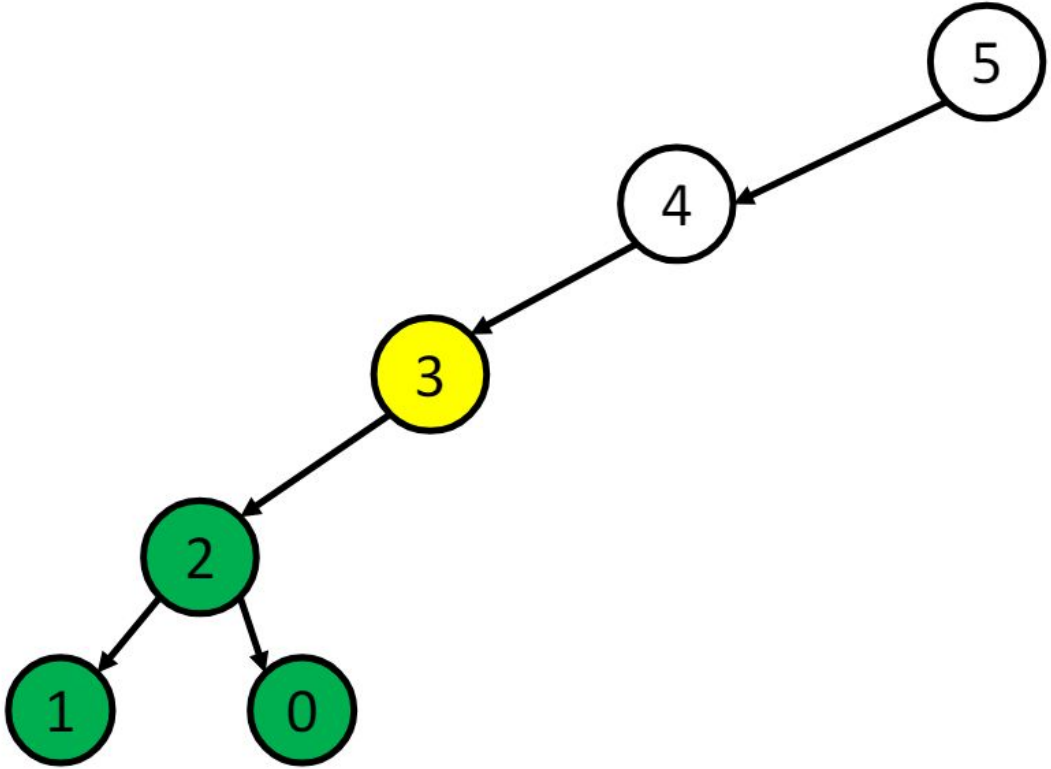

5
4
3
2

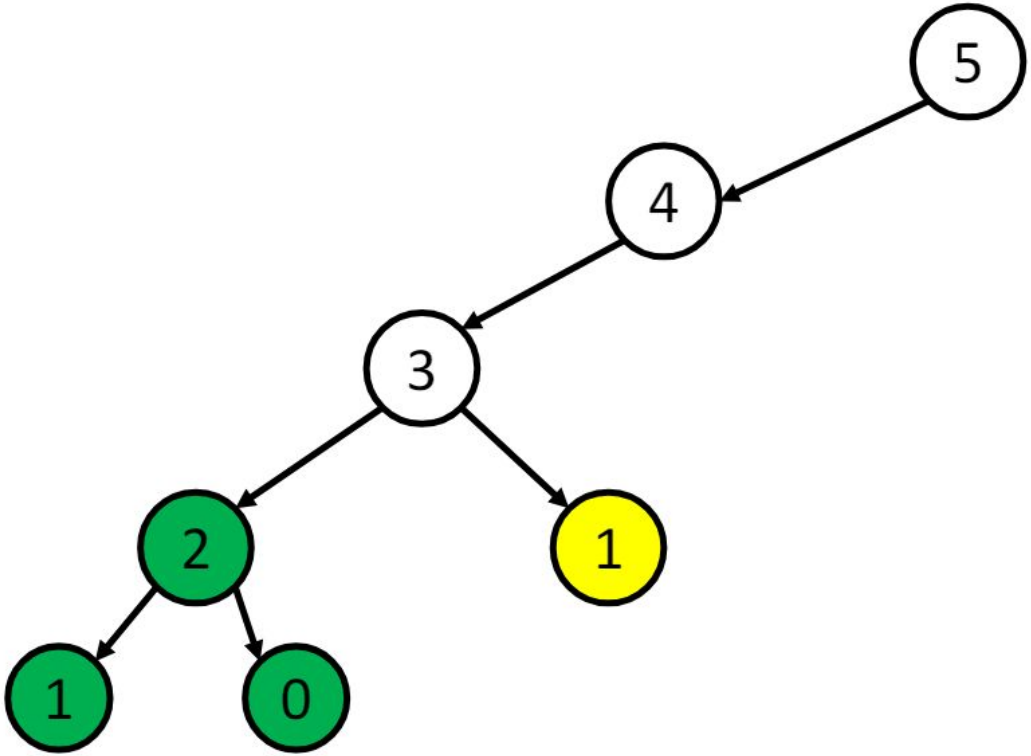

5
4
3
2
1

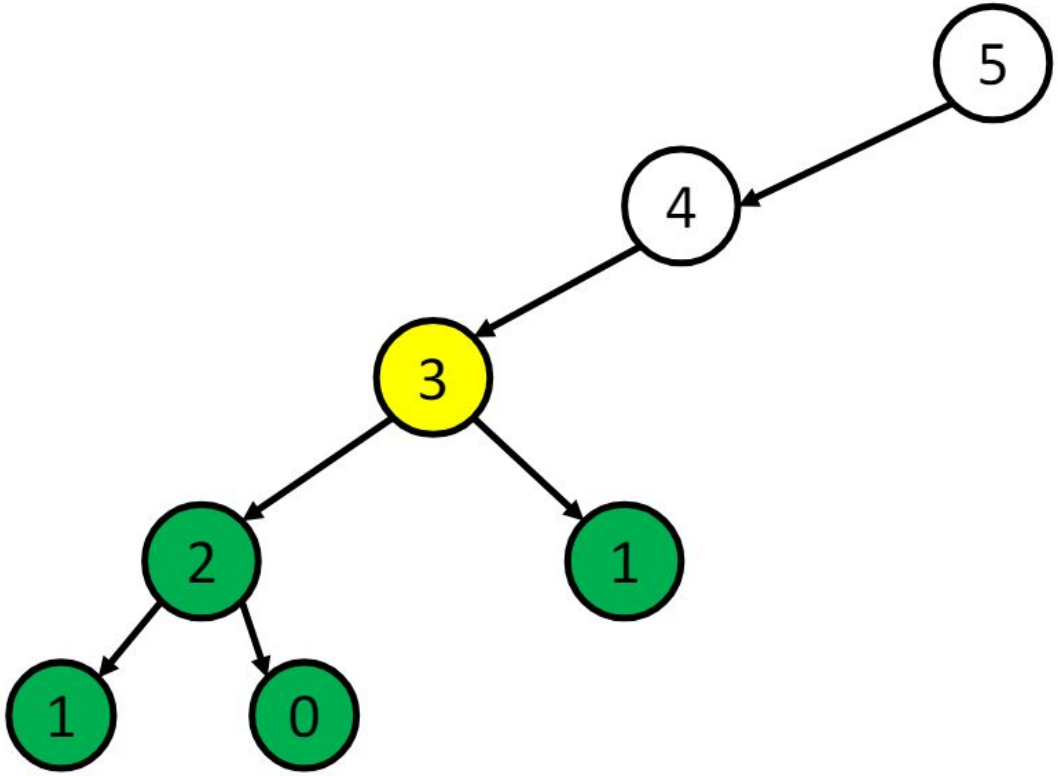

5
4
3
2
1

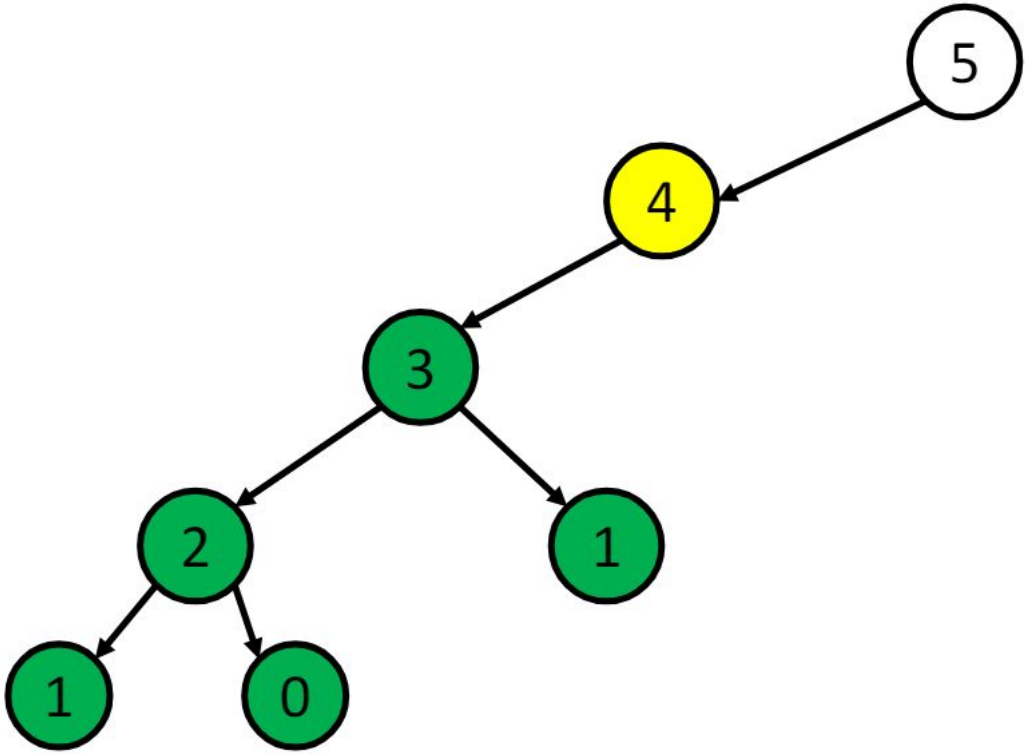

5
4
3
2
1
0

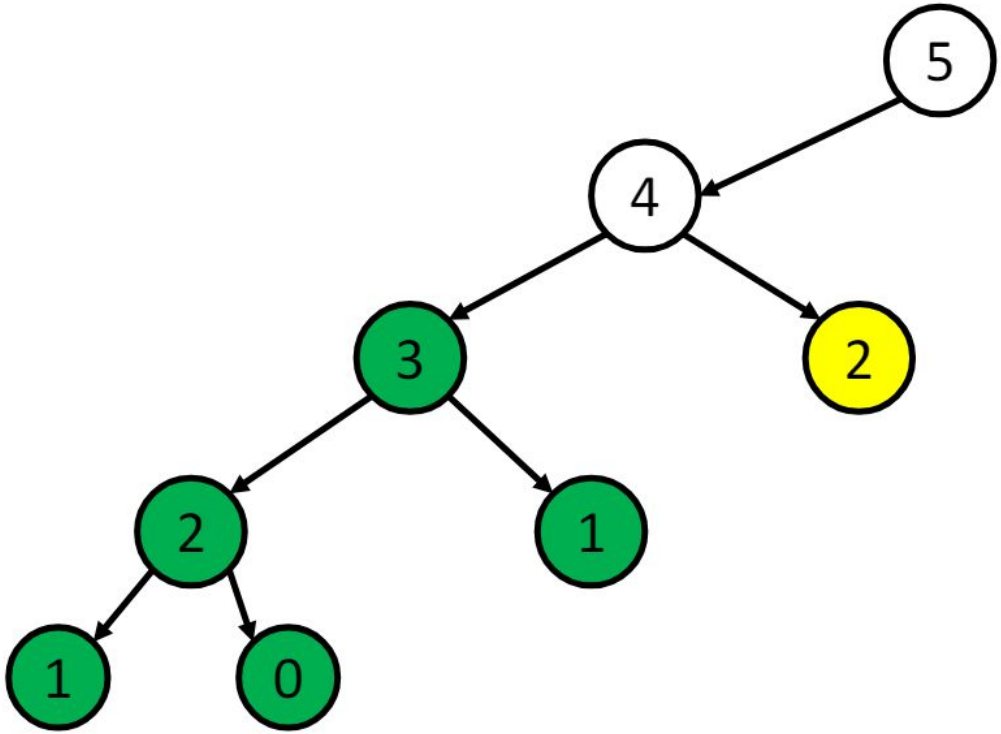

5
4
3
2
1
0

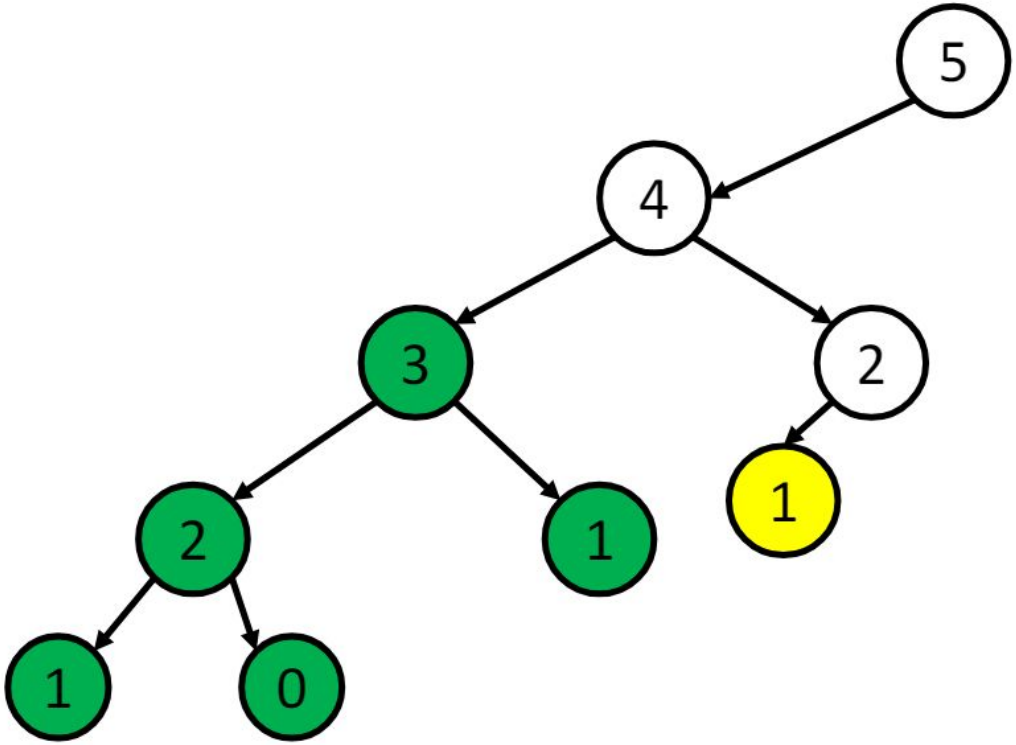

5
4
3
2
1
0

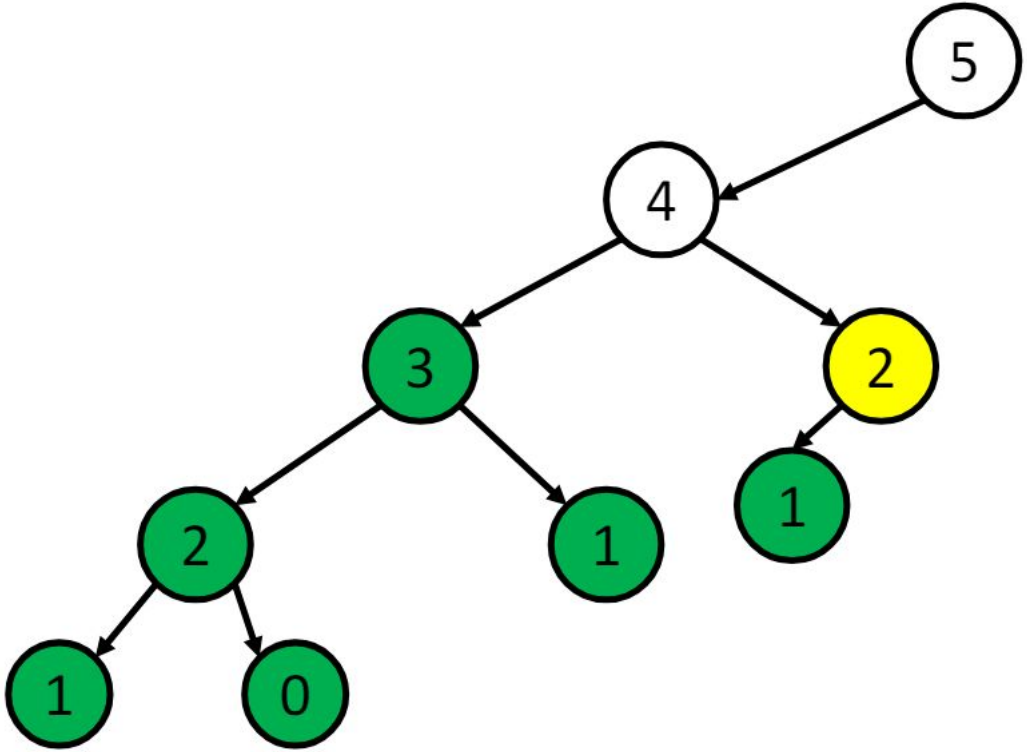

5
4
3
2
1
1
0

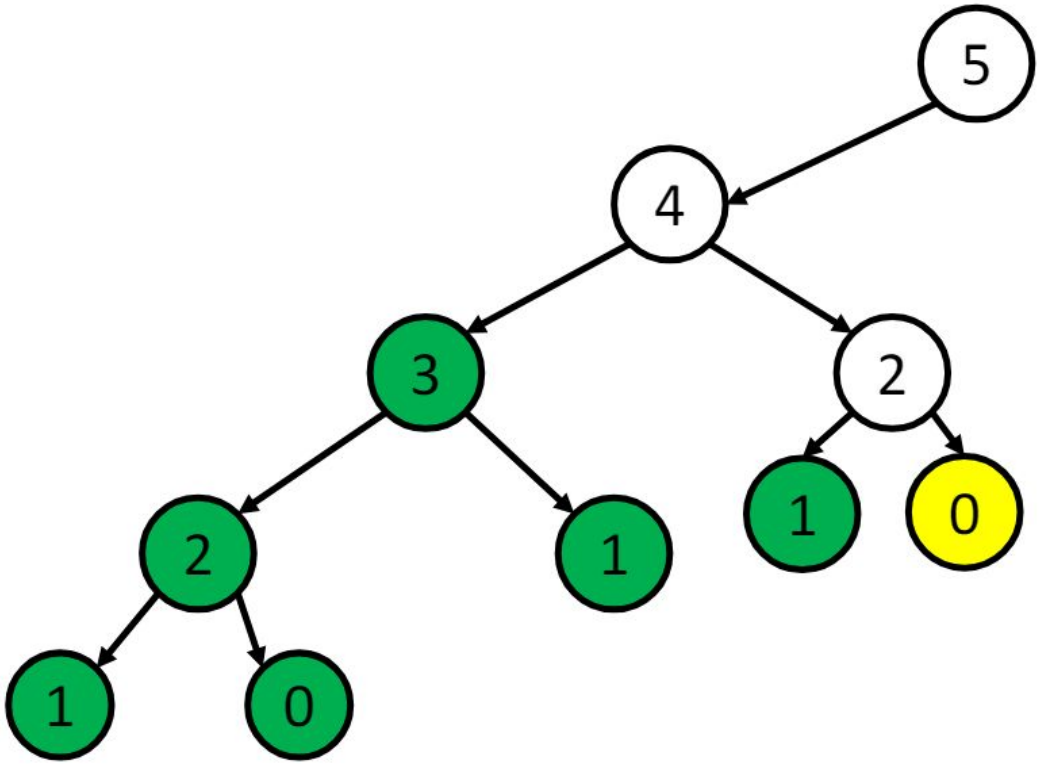

5
4
3
2
1
1
0

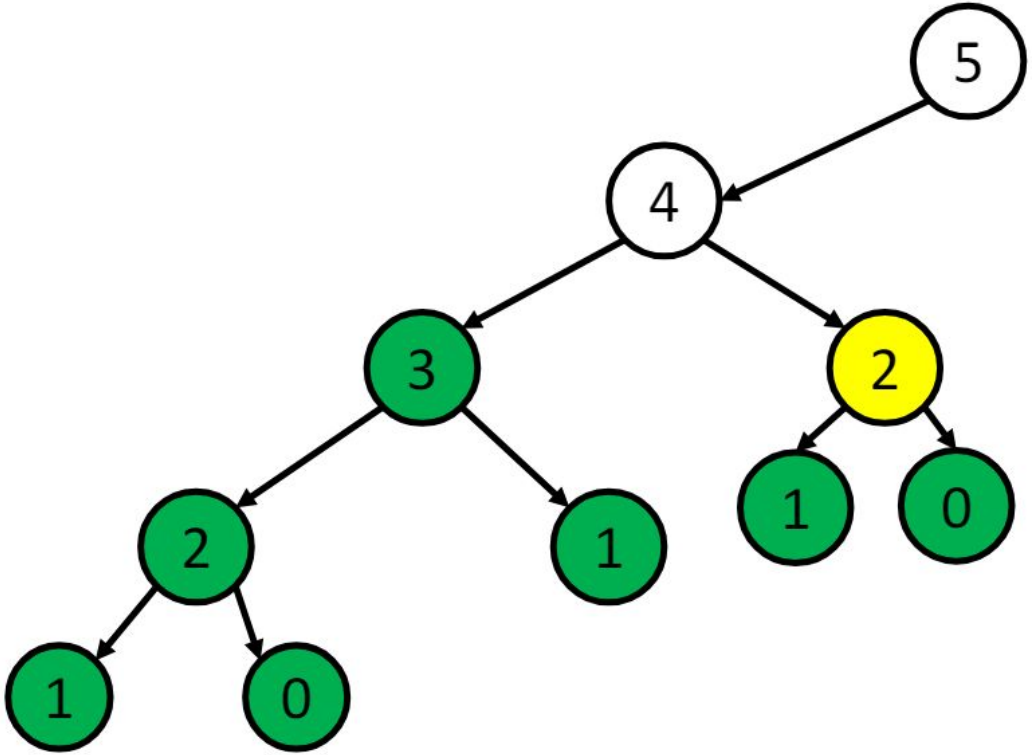

5
4
3
2
1
1
0

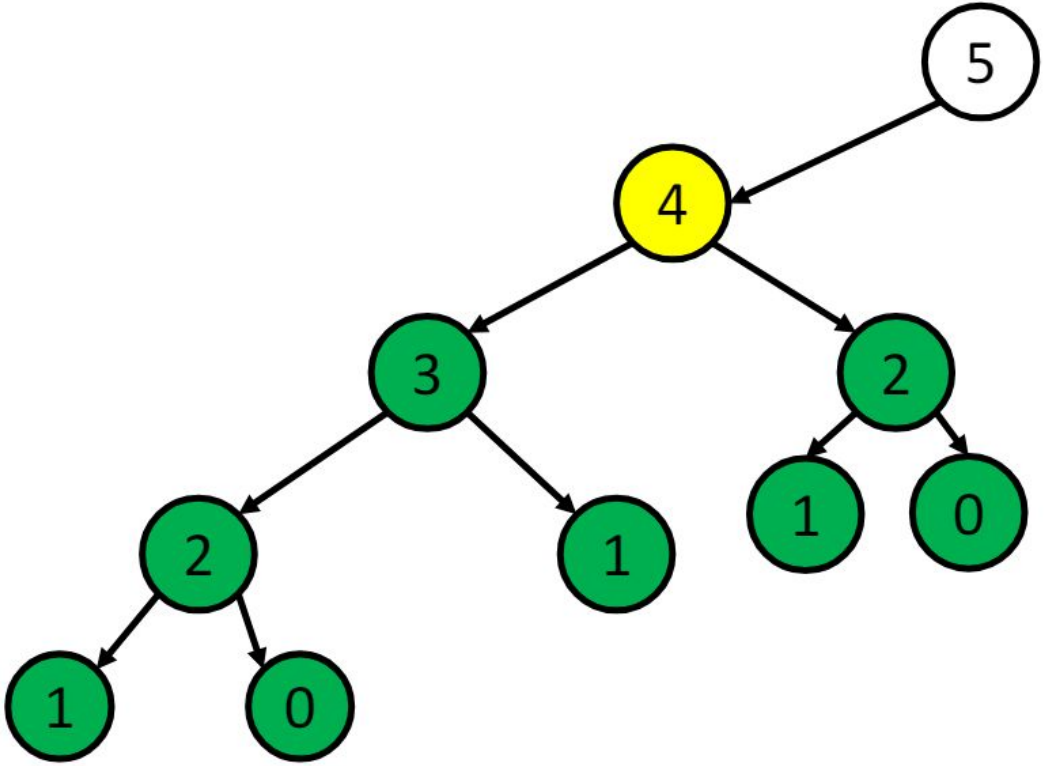

5
4
3
2
2
1
1
0

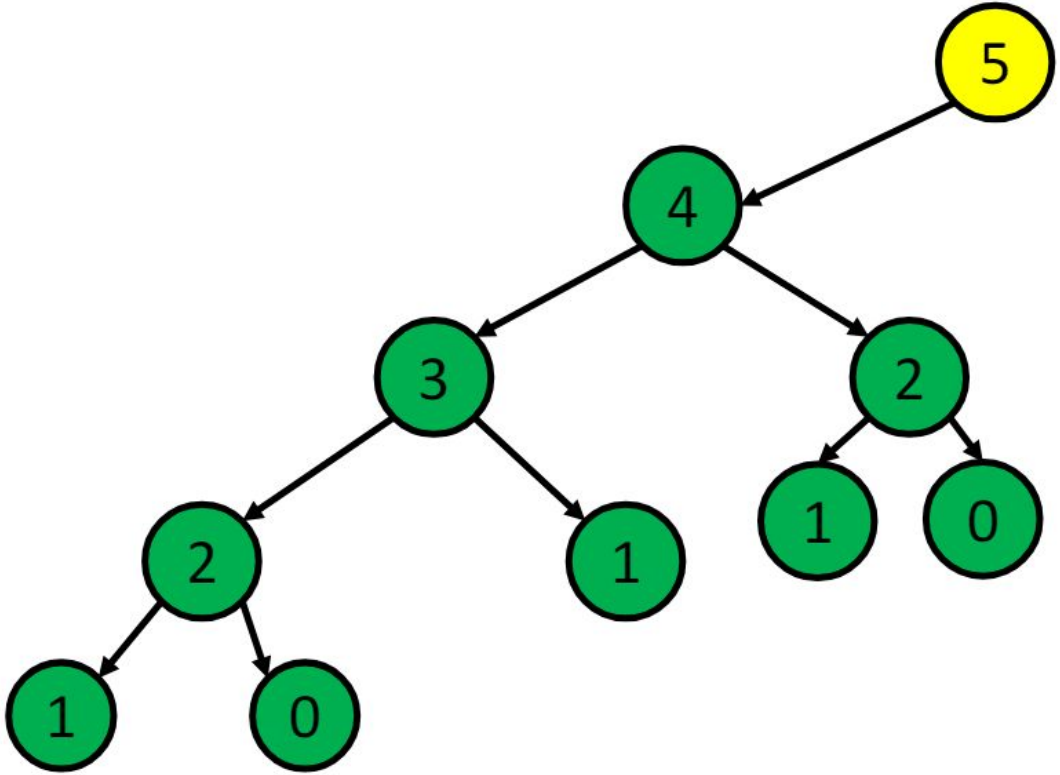

5
4
3
2
2
1
1
1
0

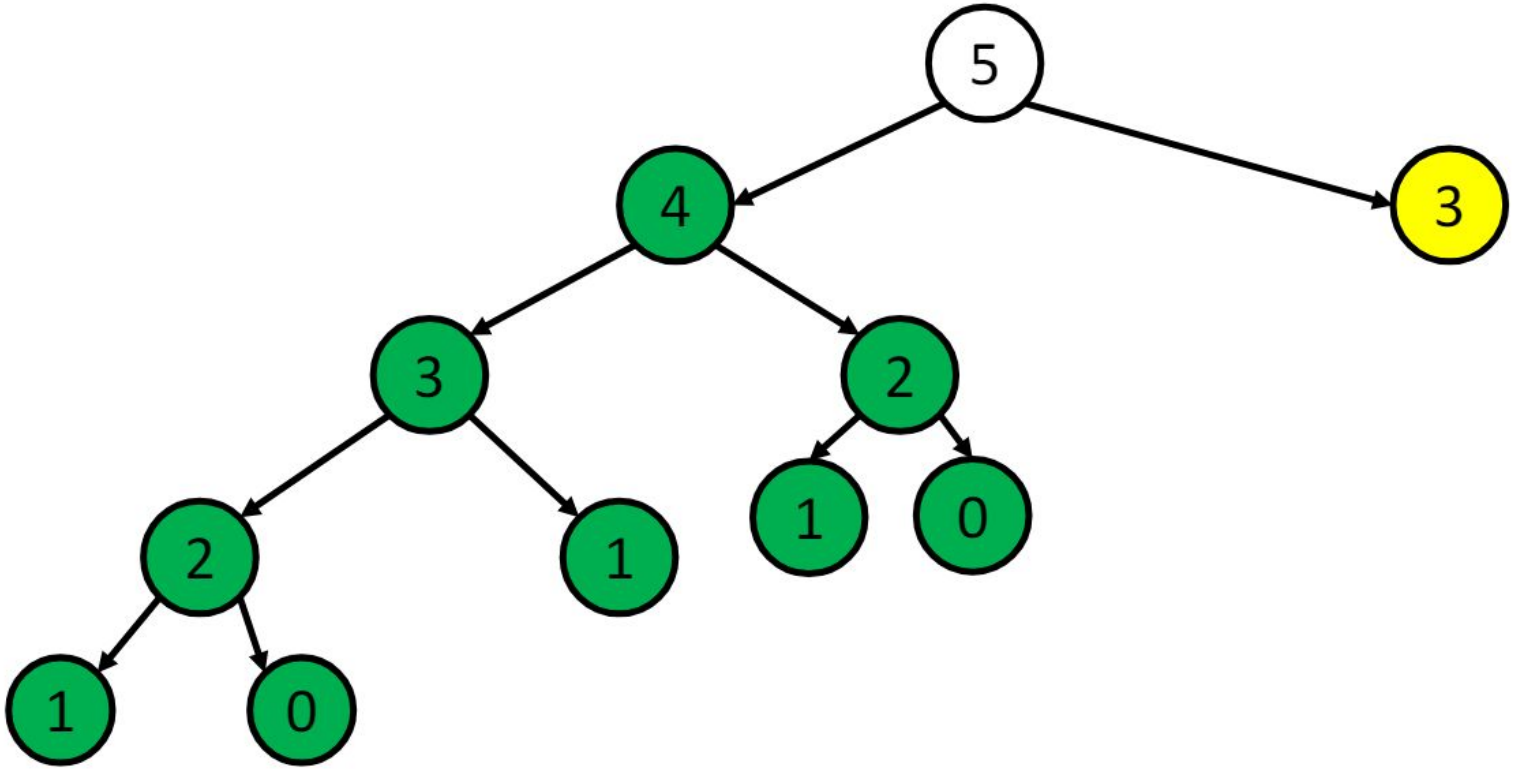

5
4
3
2
2
1
1
1
0

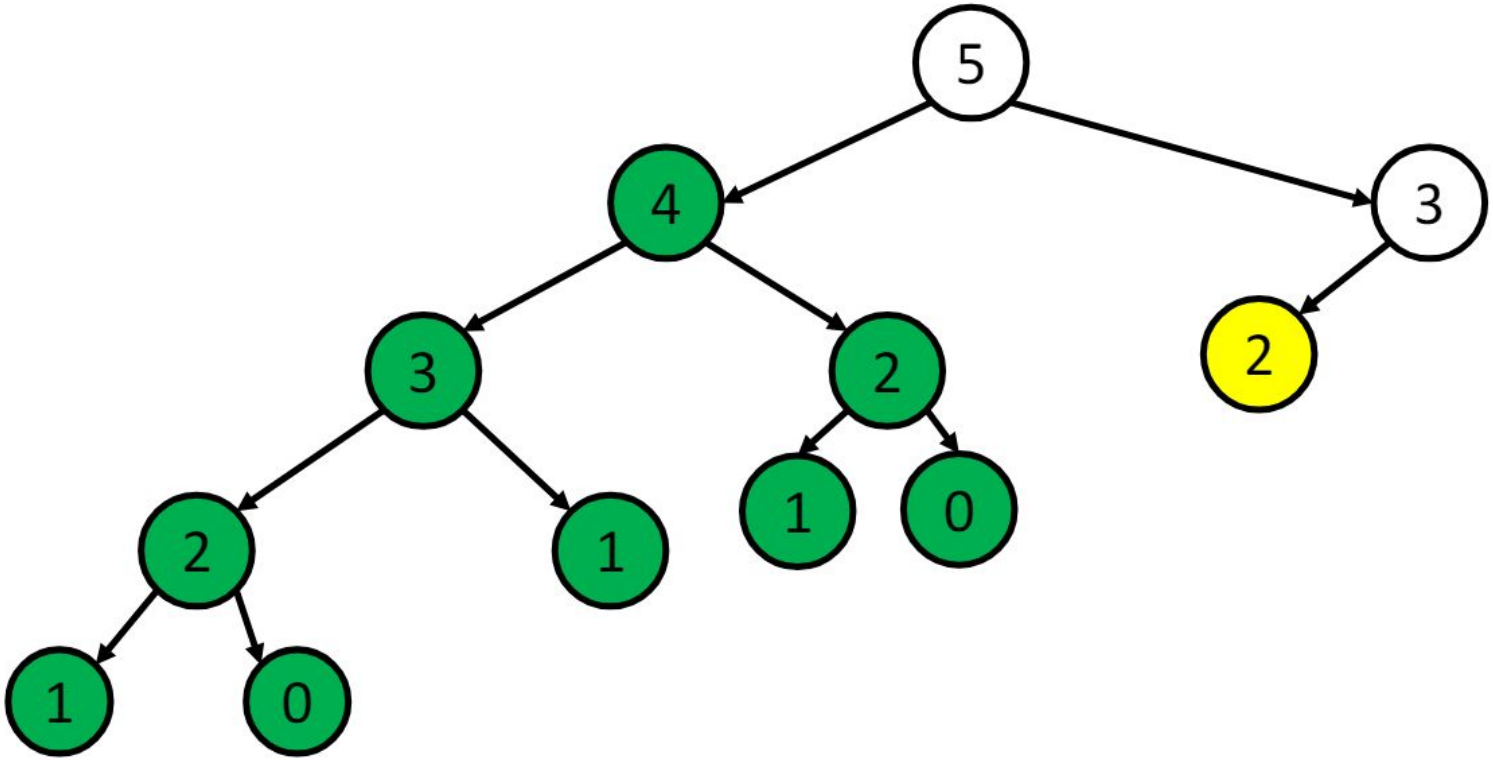

5
4
3
2
2
1
1
0
1
0

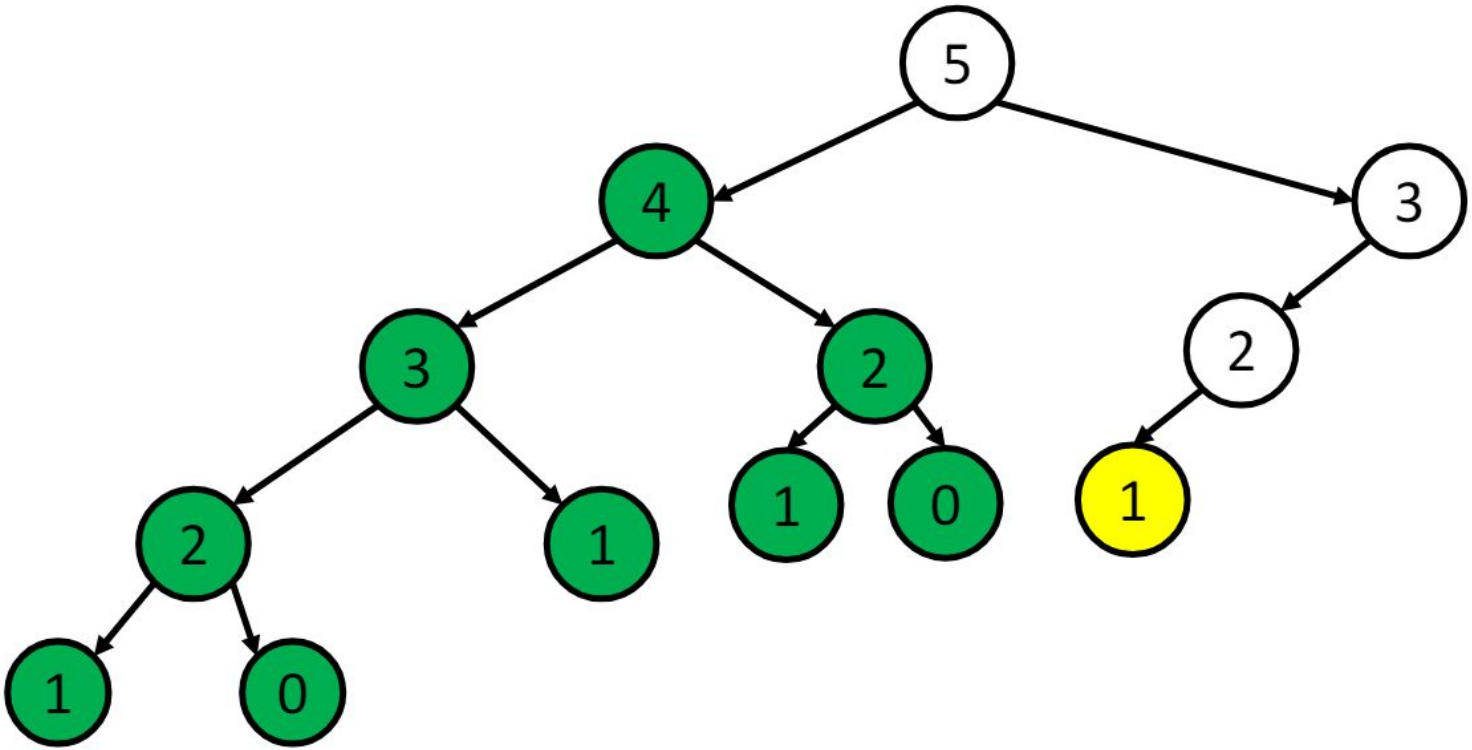

5
4
3
2
2
1
1
0
1
0

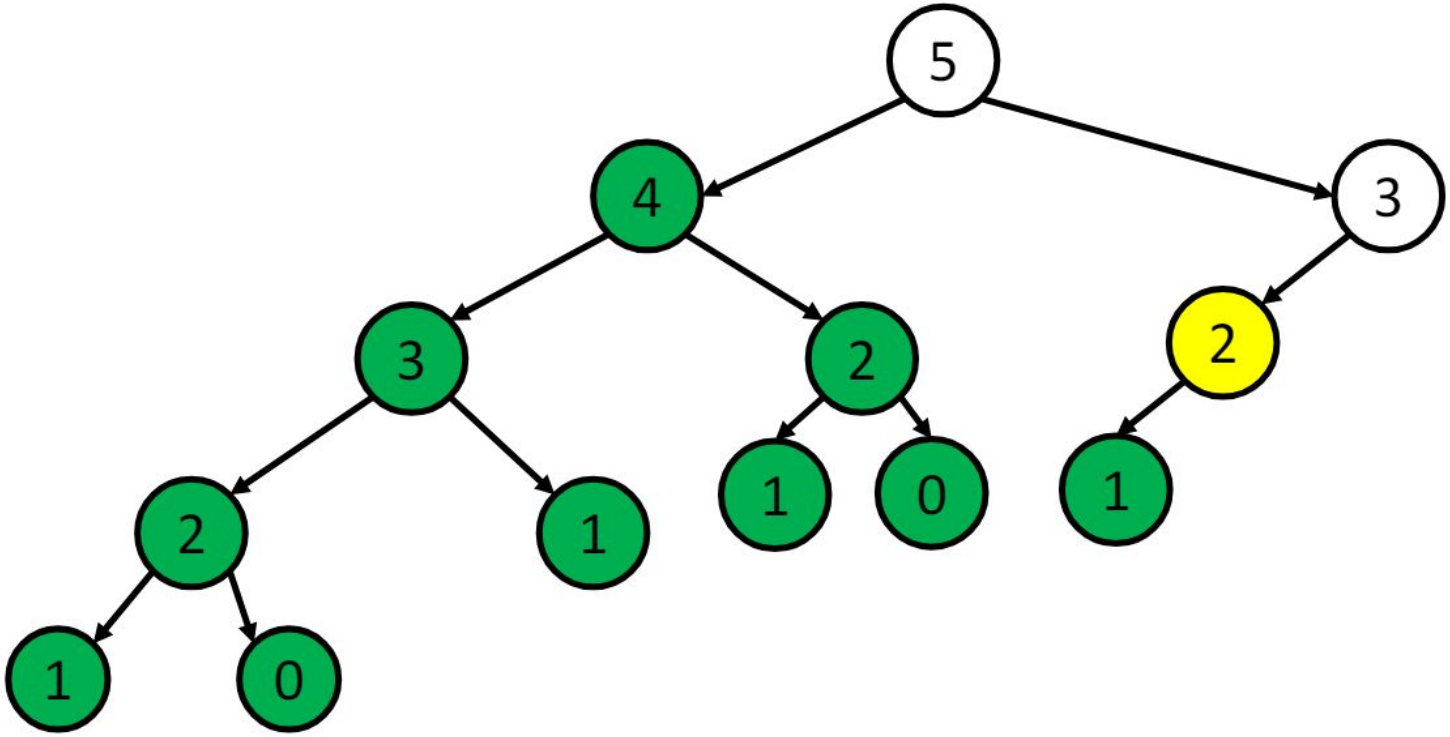

5
4
3
2
2
1
1
0
1
0

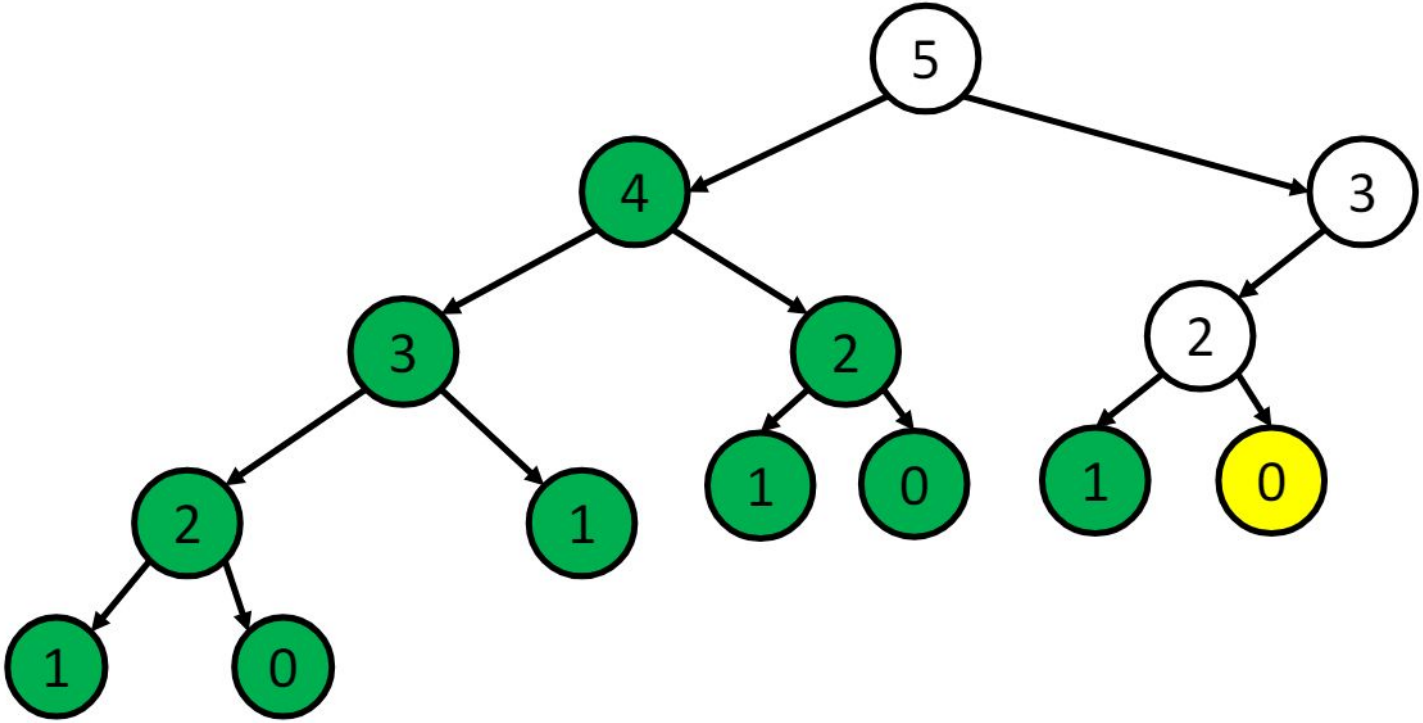

5
4
3
2
2
1
1
0
1
0

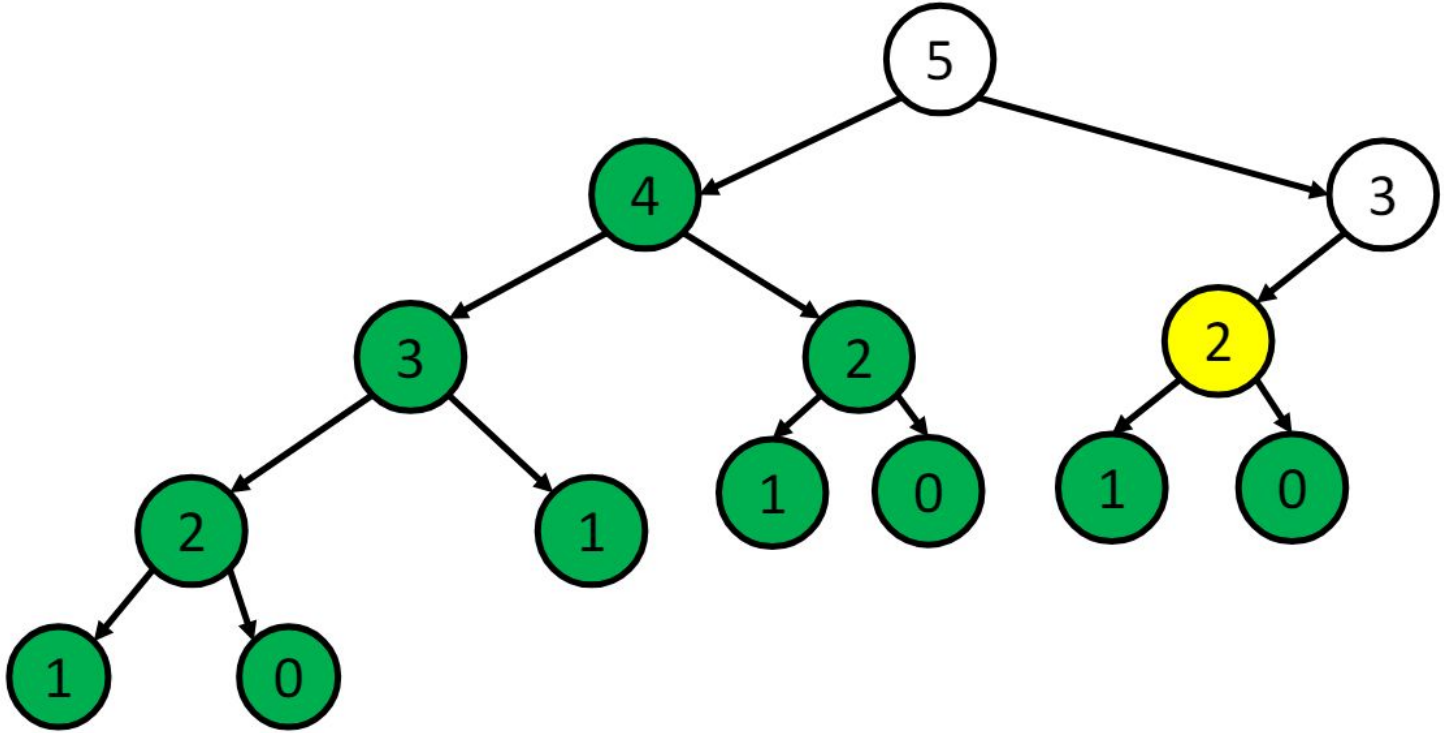

5
4
3
3
2
2
1
1
0
1
0

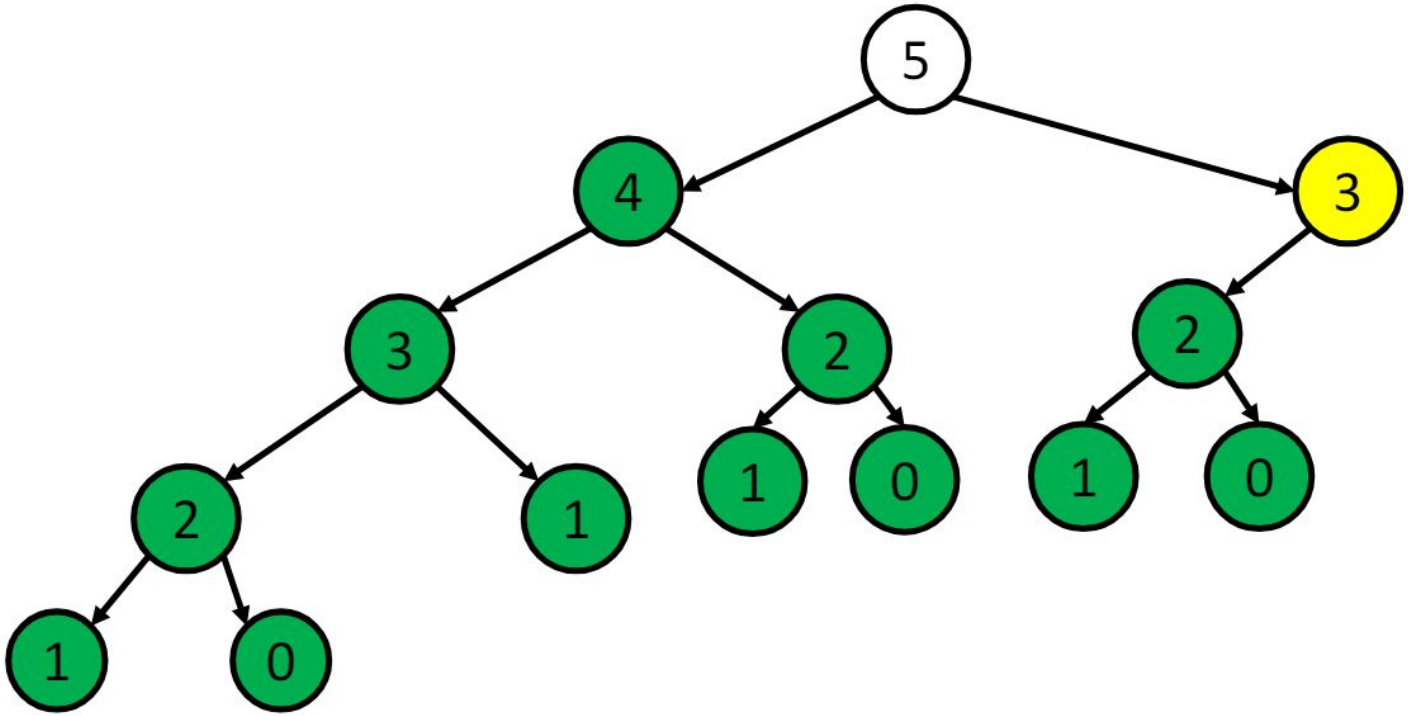

5
4
3
3
2
2
2
1
1
0
1
0

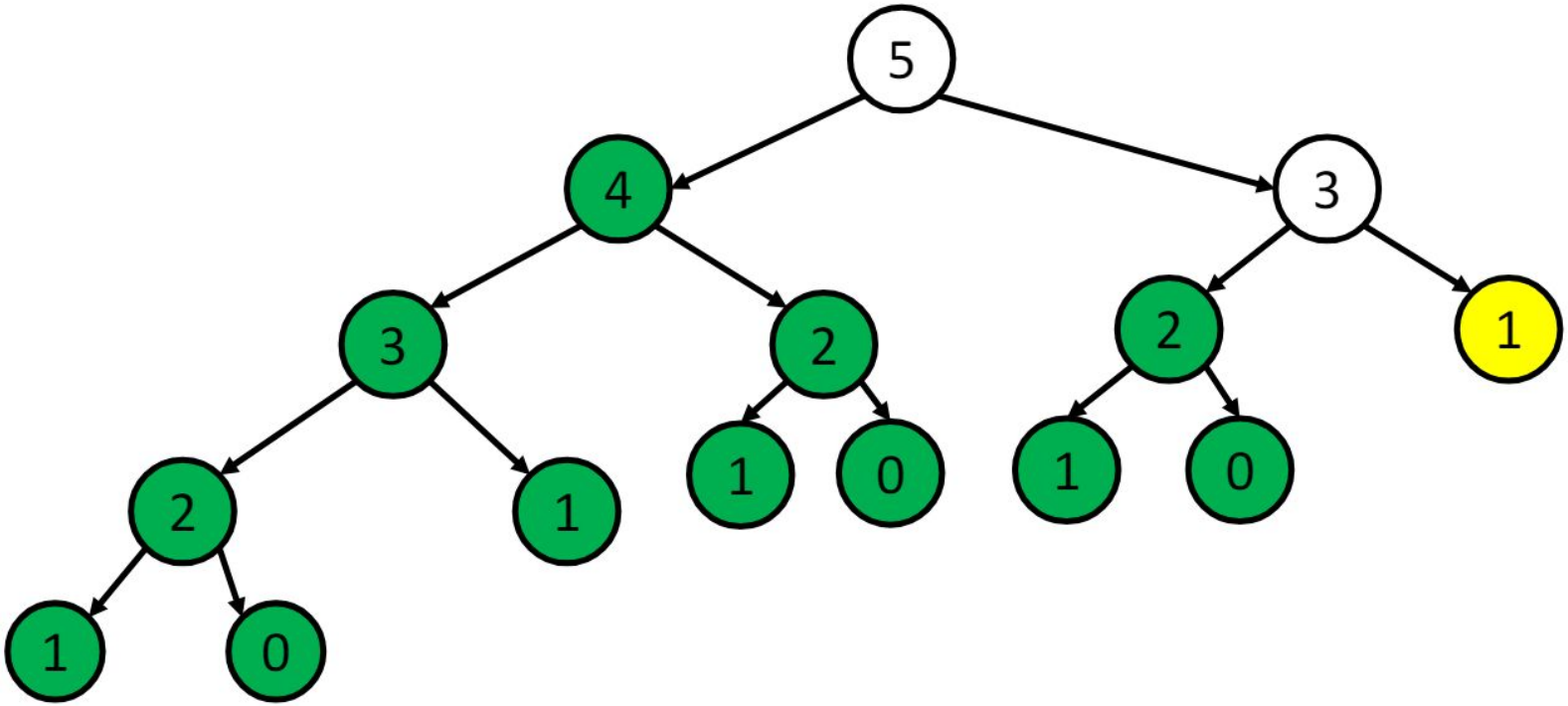

5
4
3
3
2
2
2
1
1
0
1
1
0

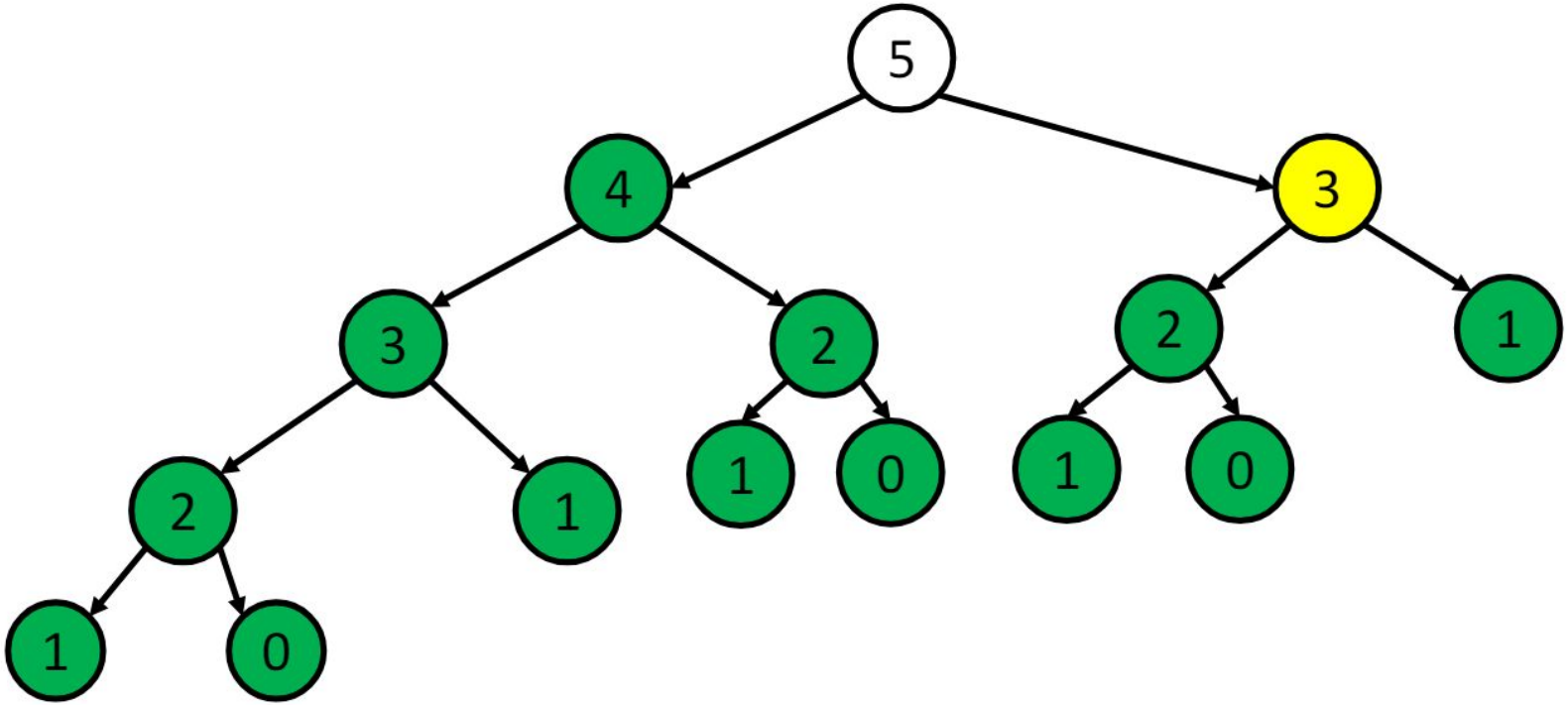

5
4
3
3
2
2
2
1
1
0
1
1
0

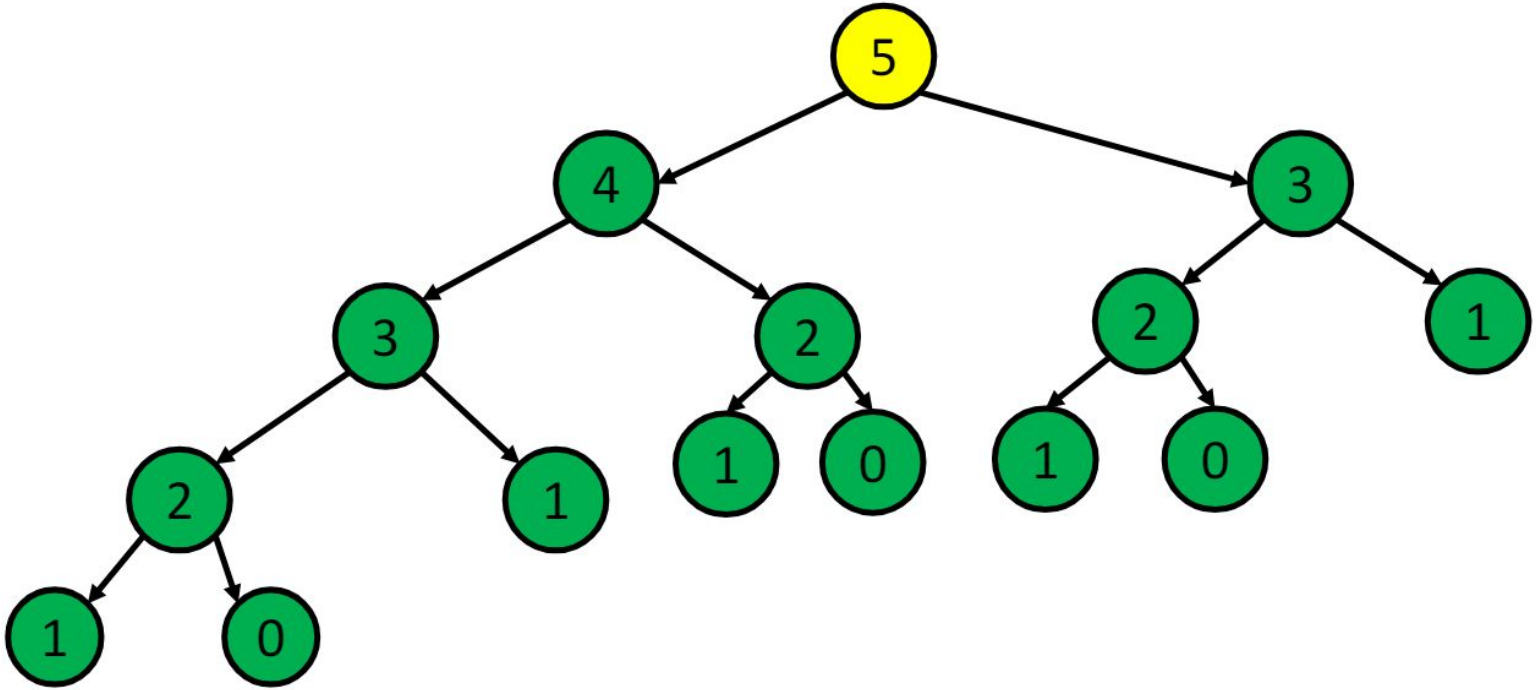

5
4
3
3
2
2
2
1
1
0
1
0
1
0

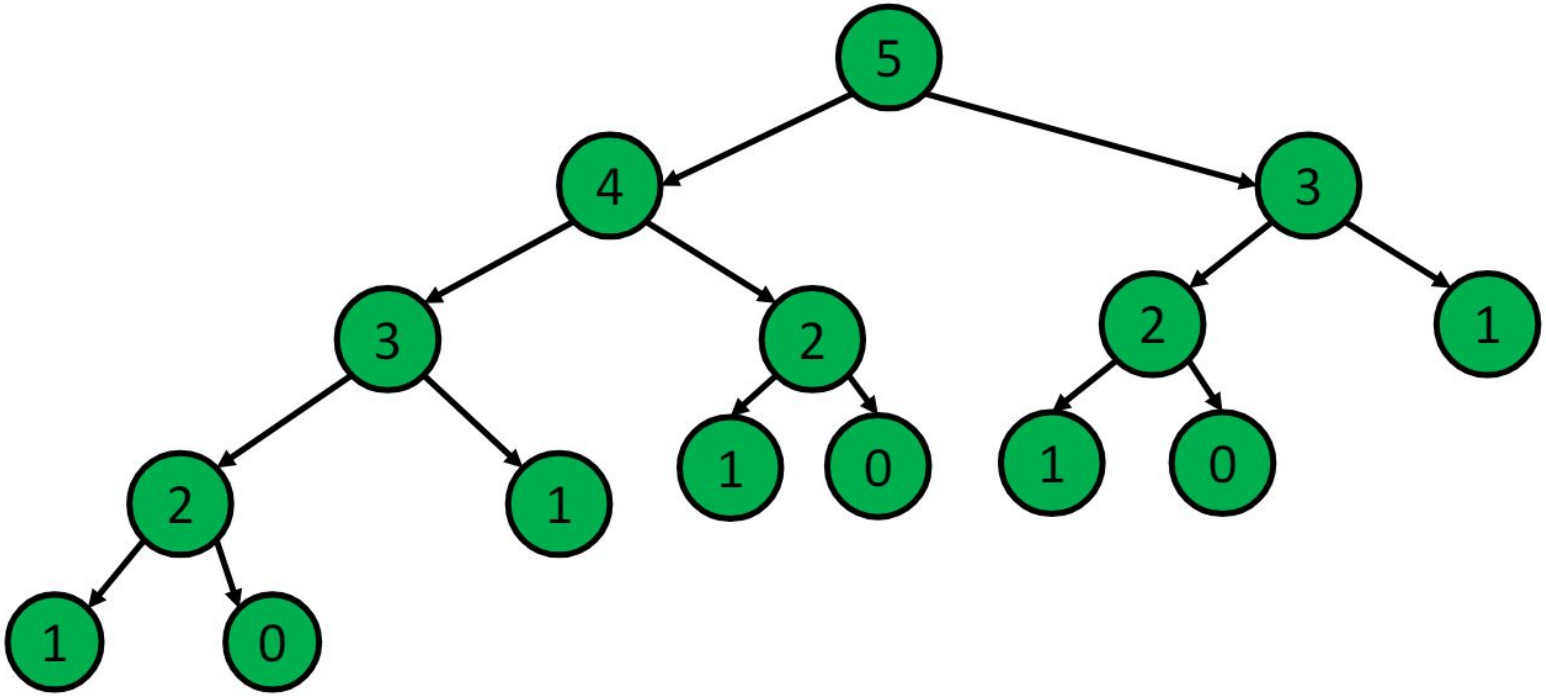

5
4
3
3
2
2
2
1
1
0
1
0
1
0


5
4
3
3
2
2
2
1
1
0
1
0
1
0


5
4
3
3
2
2
1
2
1
1
0
1
0
1
0


5
4
3
3
2
2
1
2
1
1
0
1
0
1
0


5
4
3
3
2
2
1
2
1
1
0
1
0
1
0

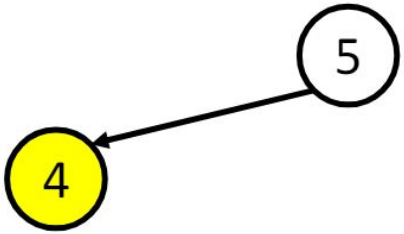

5
4
3
3
2
2
1
2
1
1
0
1
0
1
0

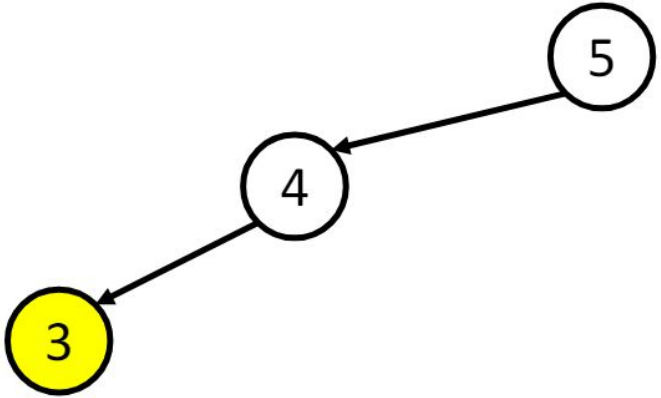
Fibonacci DP

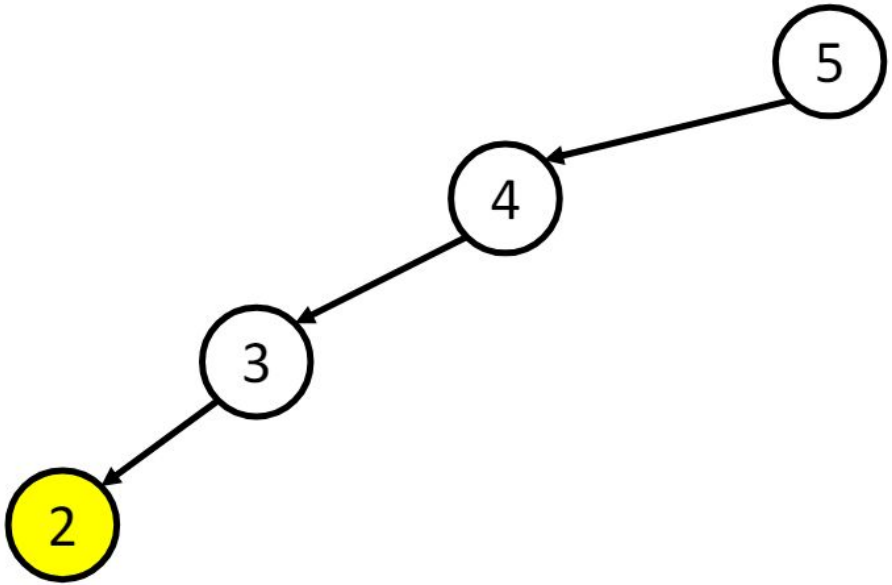
Podemos hacerlo mejor: en lugar de recalcular la respuesta cada vez, la calculamos una sola vez y guardamos el resultado O(n).

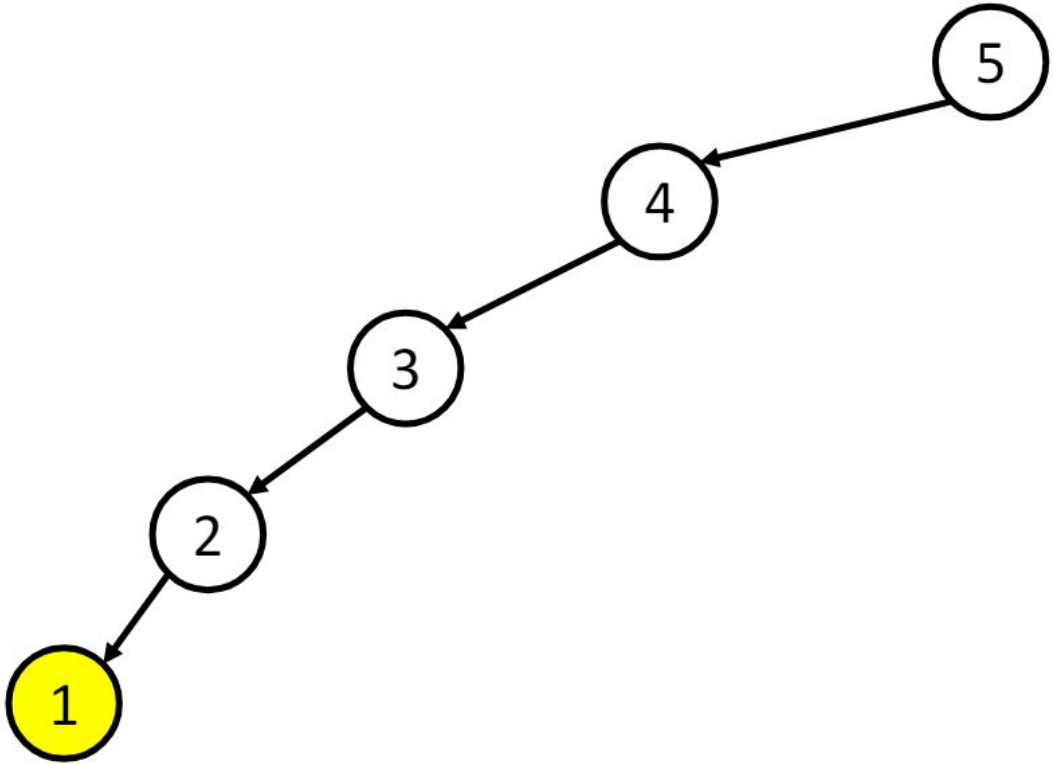
```
int dp[MAX_N];
int fib(int n) {
    if(dp[n] >= 0)
        return dp[n]
    int res;
    if(n <= 1)
        res = n;
    else
        res = fib(n -1) + fib(n -2);
    return dp[n] = res;
}
// al principio de main
fill(dp, dp + MAX_N, -1);
```

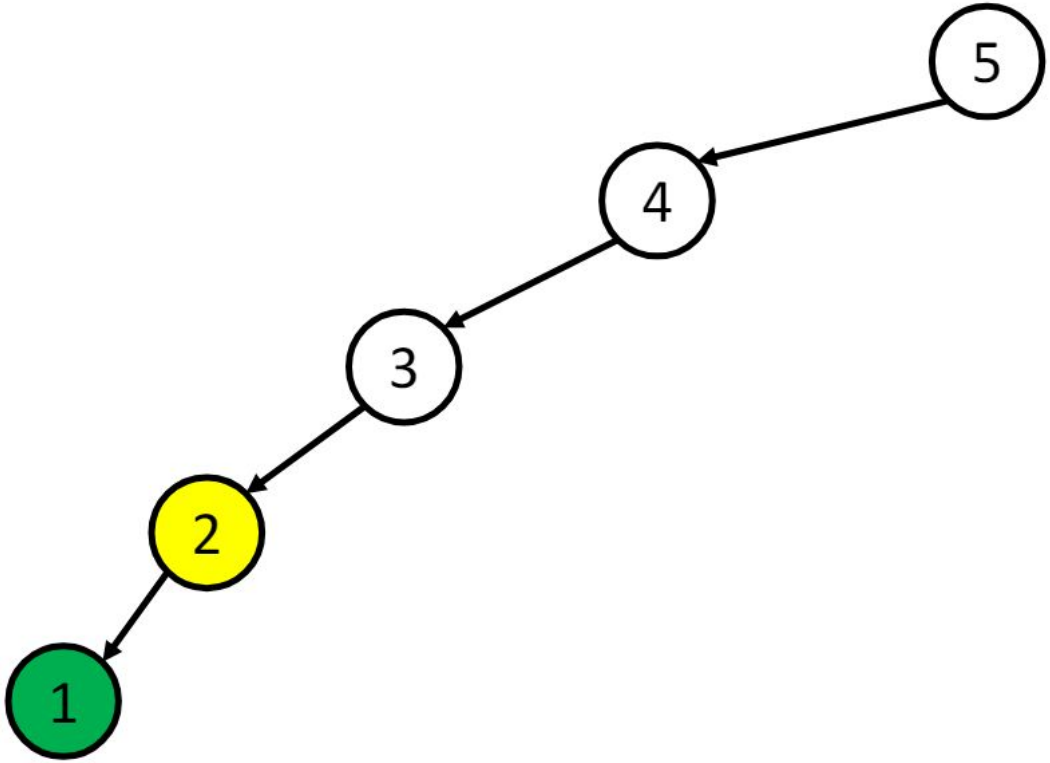
5

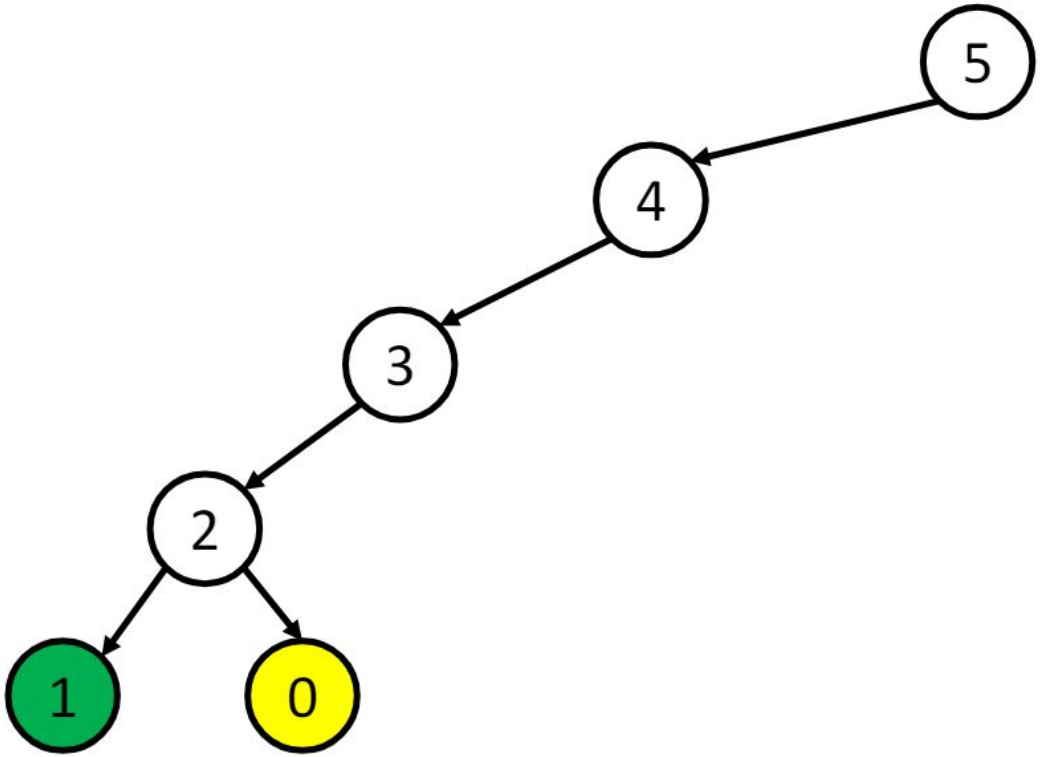

5
4

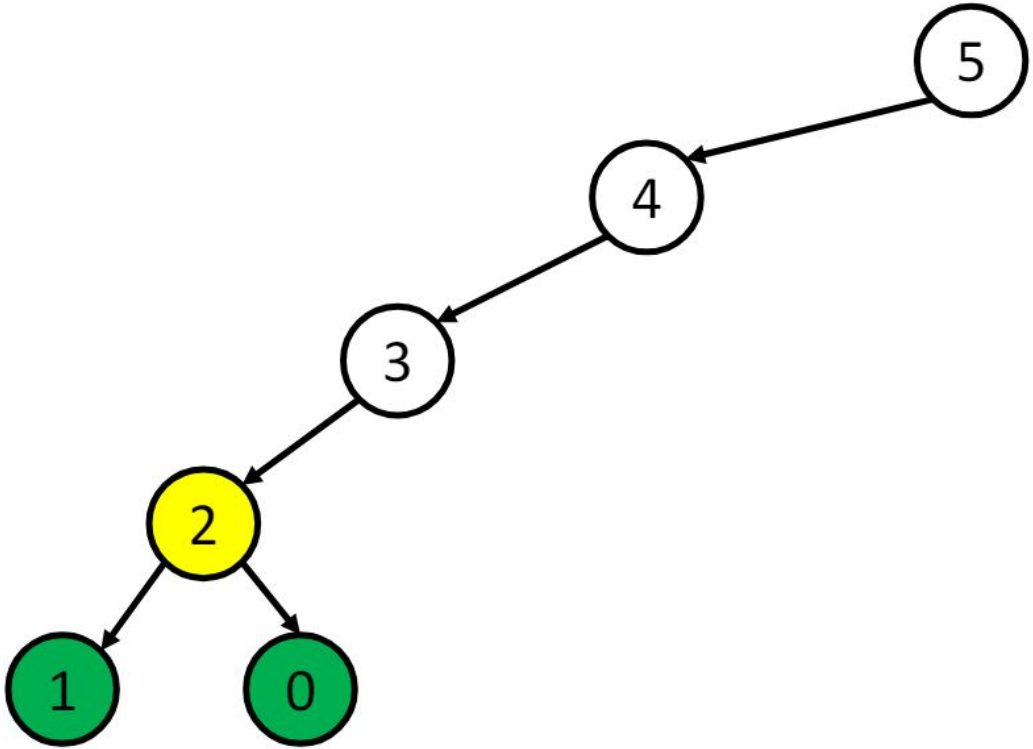

5
4
3

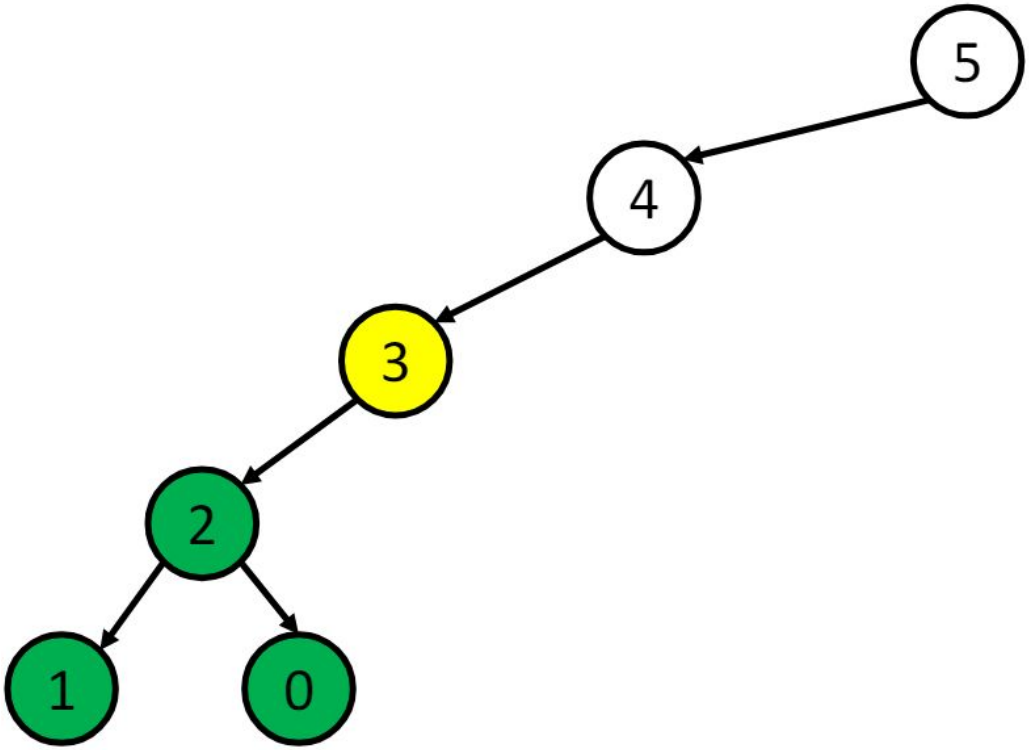

5
4
3
2

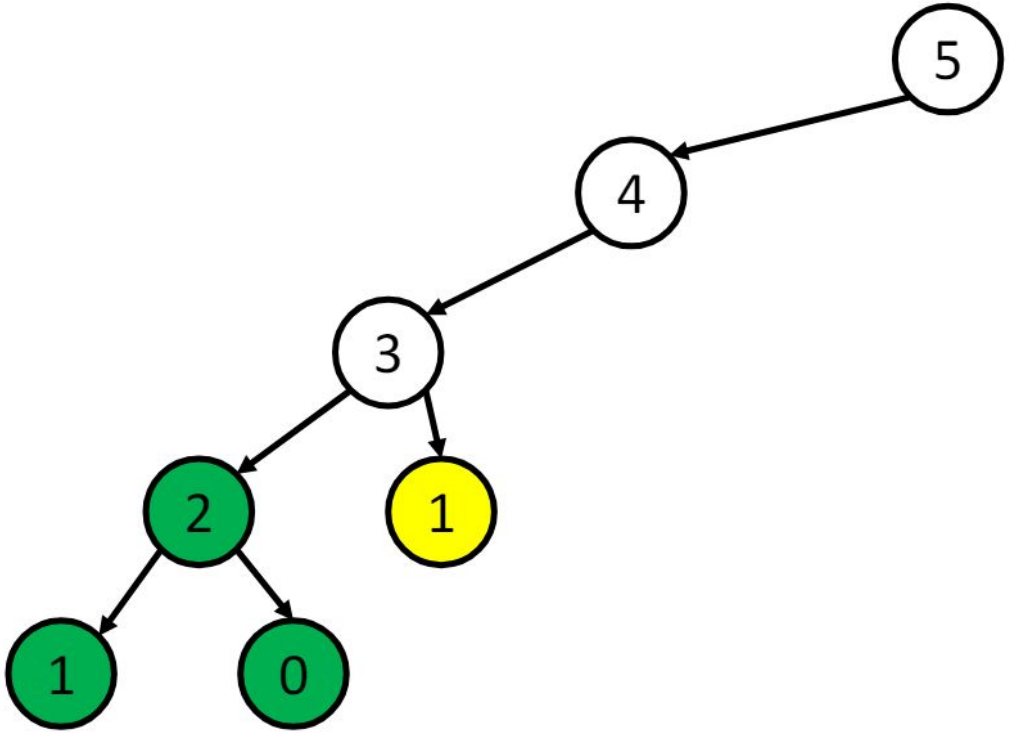

5
4
3
2
1

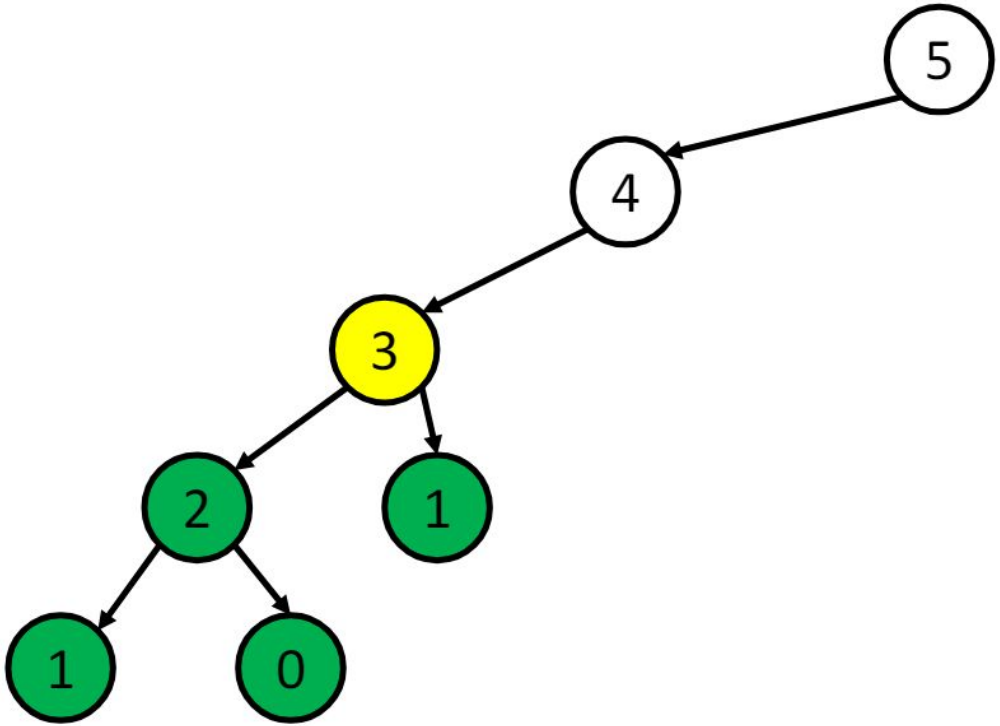

5
4
3
2
1

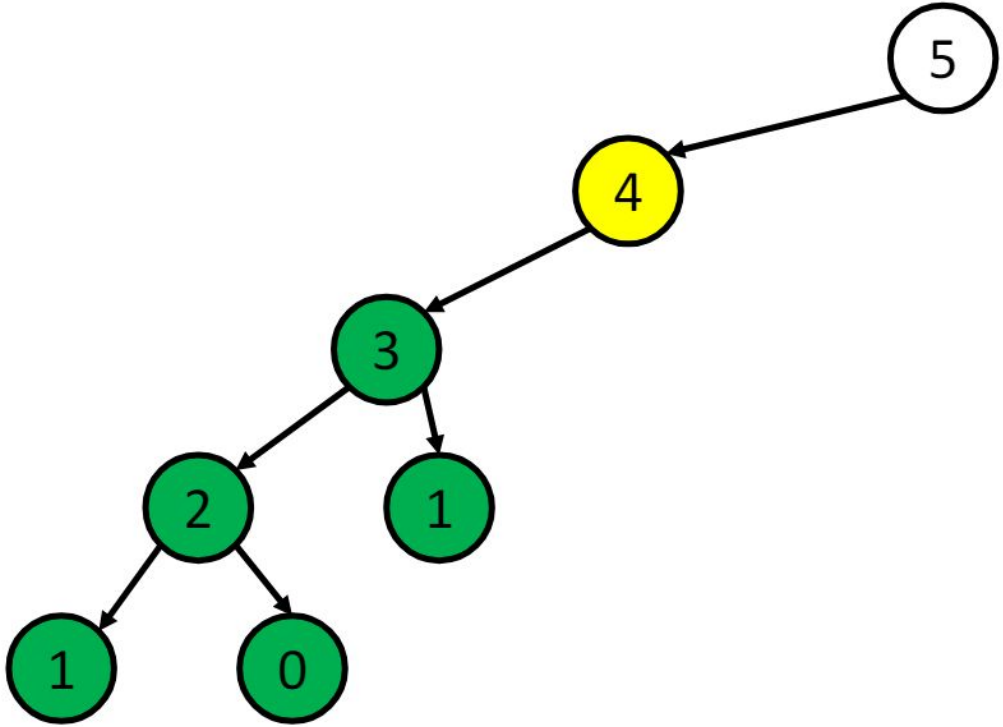

5
4
3
2
1
0

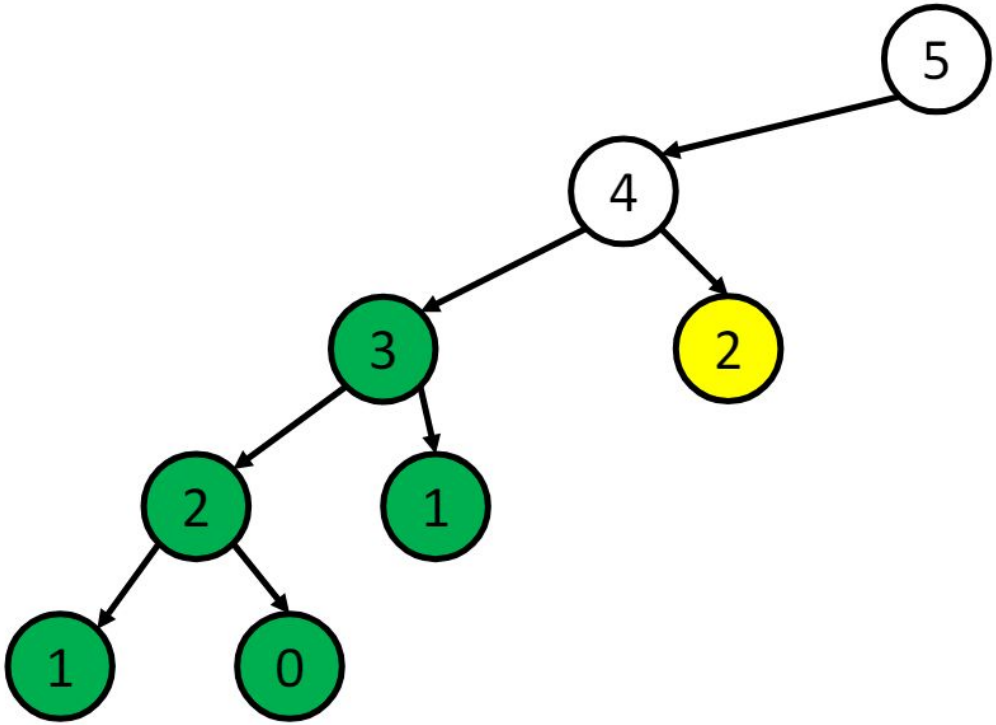

5
4
3
2
1
0

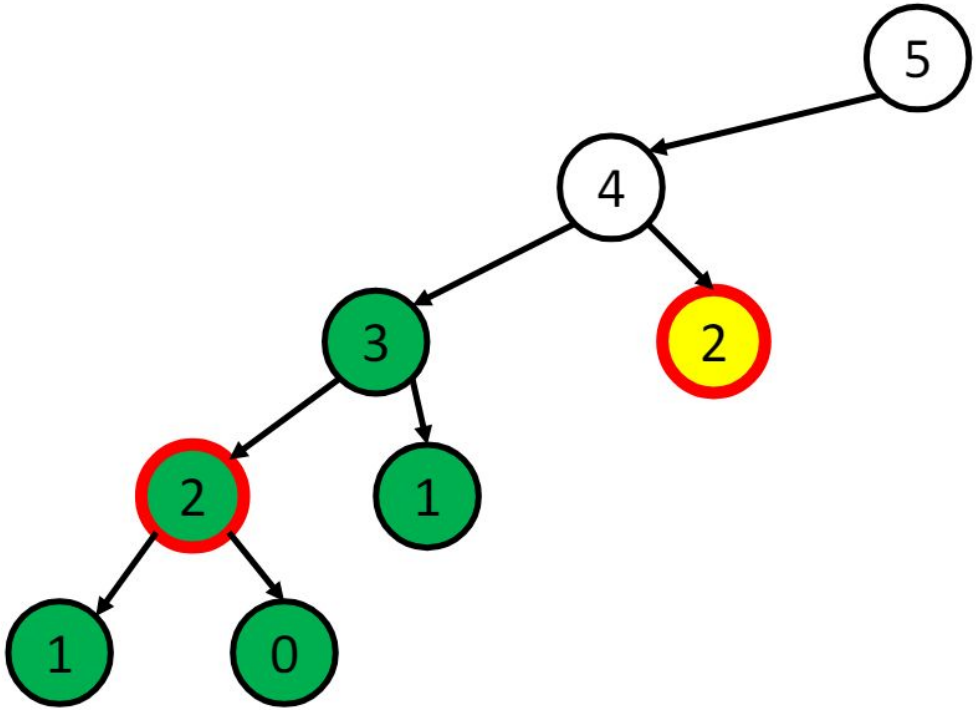

5
4
3
2
1
0

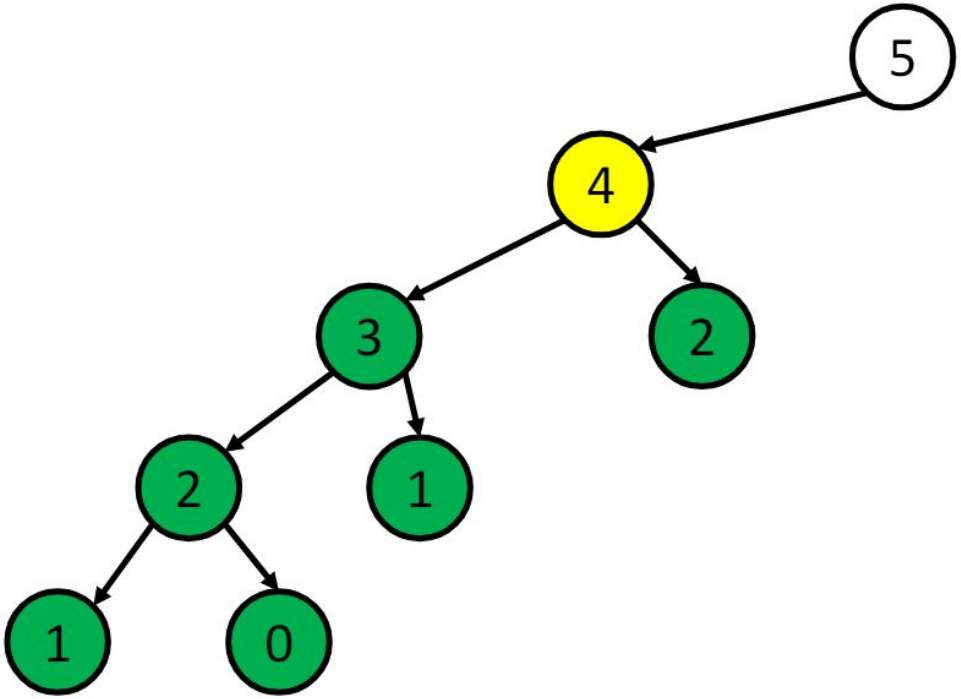

5
4
3
2
1
1
0

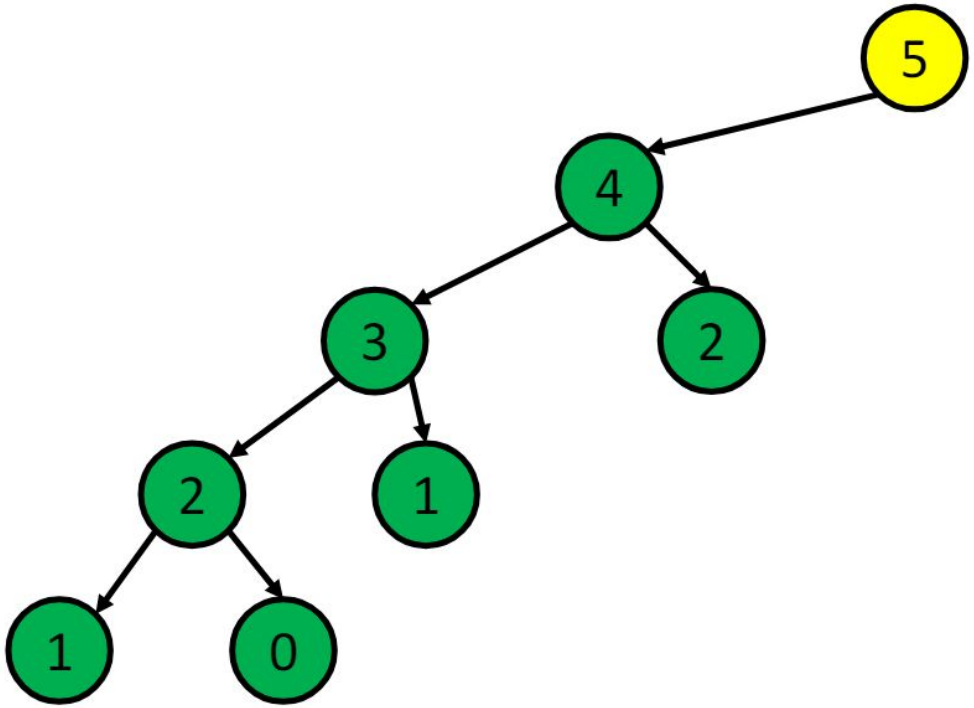

5
4
3
2
1
1
0

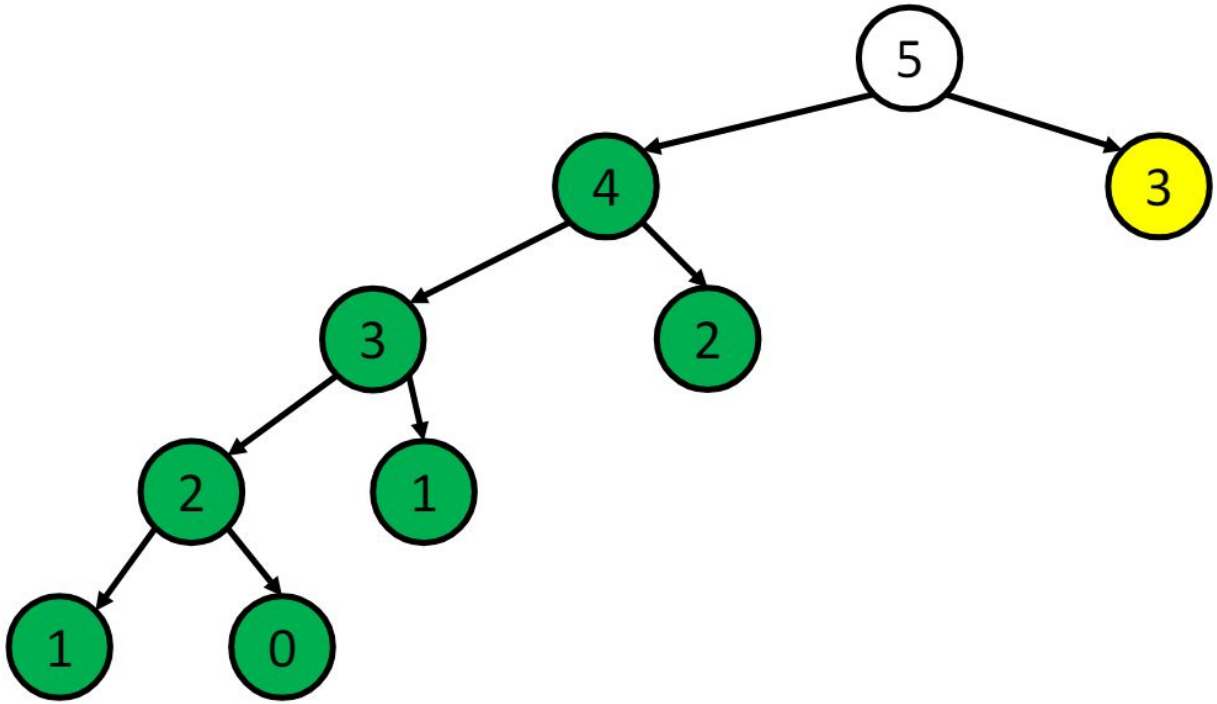

5
4
3
2
1
1
0

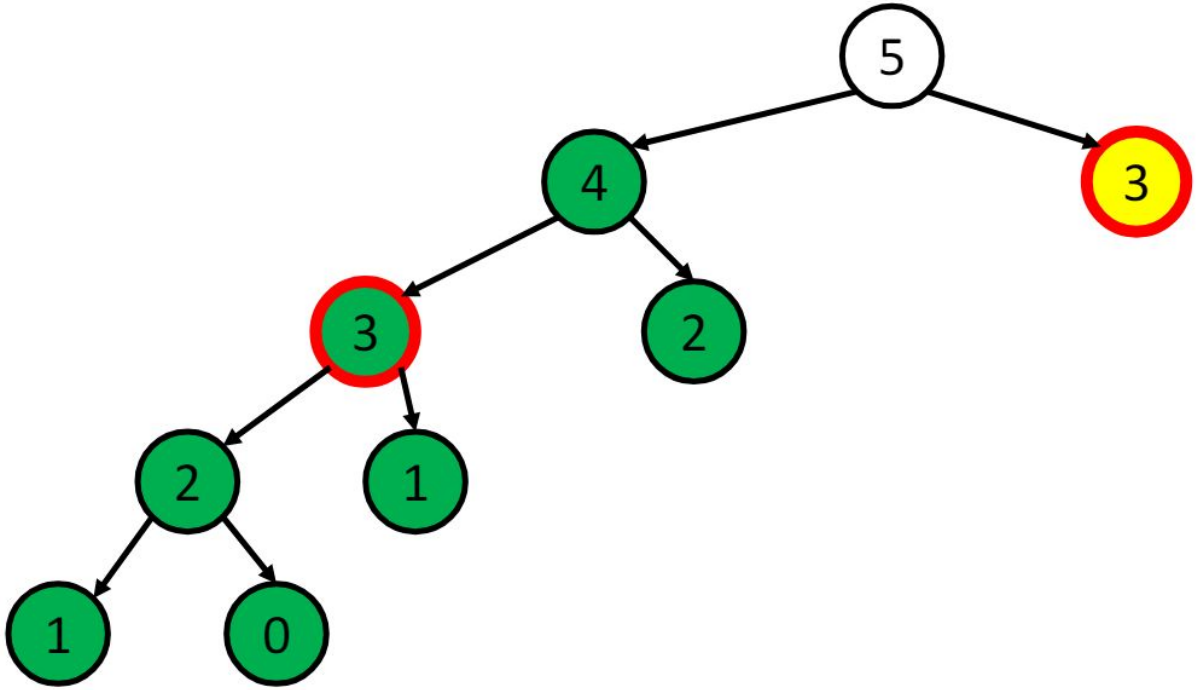

5
4
3
2
2
1
1
0

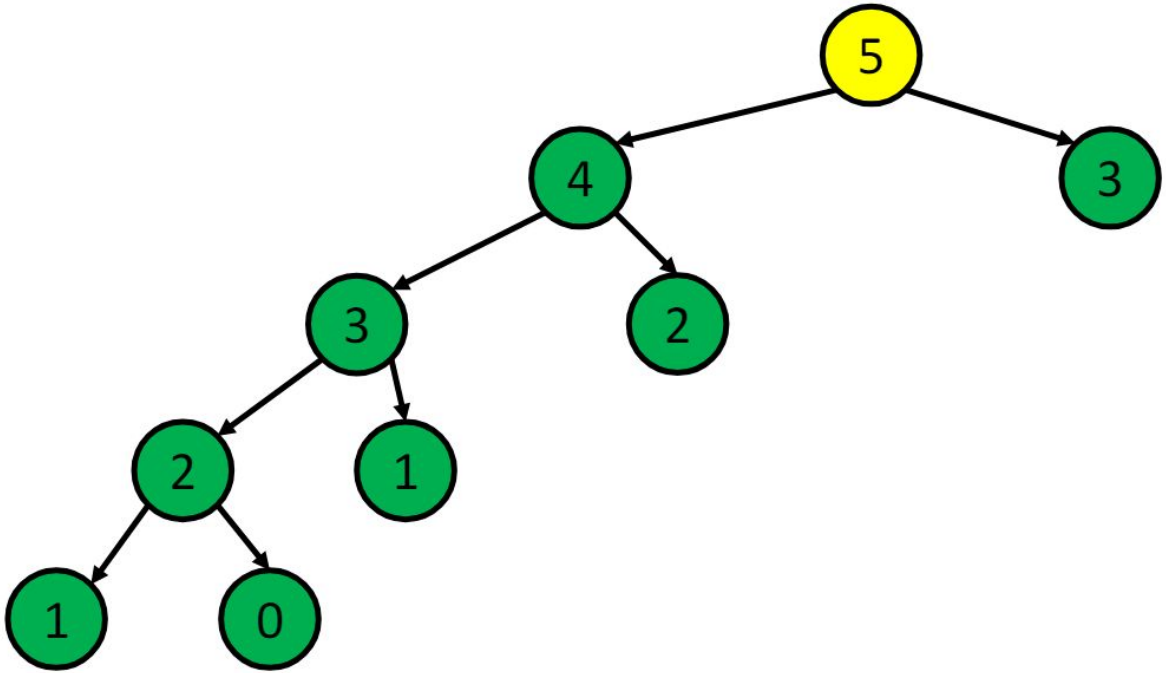

5
4
3
2
2
1
1
0

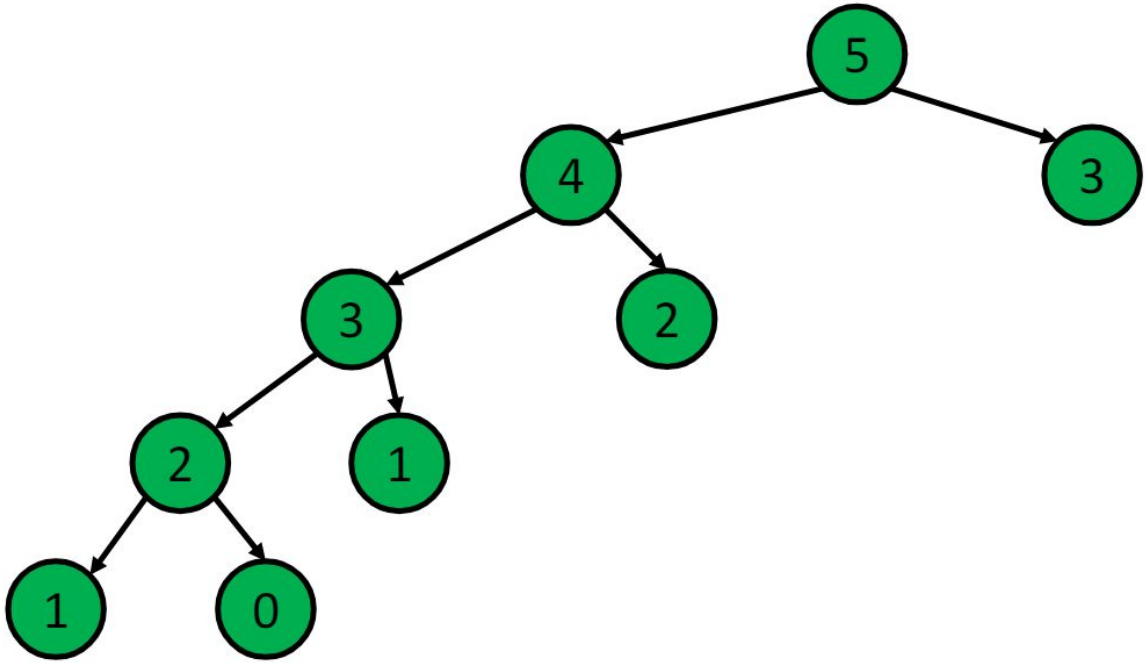

5
4
3
2
2
1
1
0


5
4
3
2
2
1
1
0


5
4
3
3
2
2
1
1
0


5
4
3
3
2
2
1
1
0


5
4
3
3
2
2
1
1
0


5
4
3
3
2
2
1
1
0

Programación Dinámica

- La programación dinámica se puede ver como una recursión con una caché para guardar resultados ya calculados.
- Podemos verla como un método para resolver un problema, que lo parte en subproblemas más pequeños, los cuáles resuelve recursivamente, guardando los resultados para no calcularlos más de una vez.
- Resuelve una gran cantidad de problemas
  - Optimizar funciones recursivas (Fibonacci, factorial, coeficiente binomial, etc.)
  - Problemas de conteo, usualmente número de formas de hacer algo
  - Minimizar o maximizar cierto valor

Programación Dinámica

- En el contexto de programación dinámica, llamamos “estado” a cada combinación de parámetros de la misma. En el caso de Fibonacci, el estado es el valor de n.
- Para calcular el costo de la programación dinámica, la cuenta suele ser “cantidad de subproblemas” * “costo de calcular el valor de un subproblema dado que los subproblemas de los cuales depende ya están calculados”
- Para realizar programación dinámica, necesitamos que no haya dependencia cíclica entre los subproblemas.

Fibonacci Iterativo

```
int dp[MAXN];
void calcFibs( int n ) {
// calcula todos los fibonaccis hasta n
// y los guarda en el arreglo dp
    dp[0] = 0;
    dp[1] = 1;
    for (int i = 2; i <= n; ++ i)
        dp[i] = dp[i - 1] + dp[i - 2];
}
```

Iterativo vs Recursivo

Tanto la forma iterativa como la recursiva tienen ventajas y desventajas.

Forma iterativa:

- Suele ser más eficiente en tiempo, porque se evitan las llamadas a función.
- A veces permite ahorrar memoria.

Forma recursiva:

- Para muchos es el modo más “natural” de pensar la programación dinámica.
- No requiere pensar en el orden en el que debemos calcular los sub-problemas, que no siempre es obvio.
- Es más eficiente cuando hay muchos valores de la tabla que no es necesario calcular (no es muy común, pero puede ocurrir).

Ejercicios

Tengo una caja de tamaño 1*N:

- es posible llenarla con bloques de 1*2?
- es posible llenarla con bloques de 1*3?
- es posible llenarla con bloques de 1*K?
- de cuántas formas se la puede llenar combinando bloques de 1*1 y 1*2?
- de cuántas formas se la puede llenar combinando bloques de 1*1 y 1*K?
- de cuántas formas se la puede llenar combinando bloques de 1*K y 1*L?
- y si es posible dejar espacios vacíos?

CSES 1633 - Dice Combinations

contar el número de formas de construir la suma n lanzando un dado una o más veces. Cada lanzamiento produce un resultado entre 1 y 6.

n <= 10^6

Por ejemplo, si n=3, hay 4 maneras:

- 1+1+1
- 1+2
- 2+1
- 3

Código Iterativo

```
int n;
cin>>n;
vi dp(n + 1);
dp[0] = 1;
fore(i, 1, n + 1)
    fore(j, 1, 7)
        if(j <= i)
            dp[i] = (dp[i] + dp[i - j]) % MOD;
cout<<dp[n]<<'\n';
```

Código Recursivo

```
vi dado({1, 2, 3, 4, 5, 6});
bool vis[tam];
int dp[tam];
int f(int suma)
{
    if(suma < 0) return 0;
    if(suma == 0) return 1;
    if(vis[suma]) return dp[suma];
    vis[suma] = 1;
    for(int x : dado)
        dp[suma] = (dp[suma] + f(suma - x)) % MOD;
    return dp[suma];
}
f(n);
```

CSES 1634 – Minimizing Coins

Consideremos un sistema monetario formado por n monedas. Cada moneda tiene un valor entero positivo. Su tarea es producir una suma de dinero x utilizando las monedas disponibles de tal manera que el número de monedas sea mínimo.

Por ejemplo, si las monedas son {1,5,7} y la suma deseada es 11, una solución óptima es 5+5+1, que requiere 3 monedas.

Imprimir la menor cantidad de monedas necesarias para sumar x

1 <= n <= 100, 1 <= x <= 10^6, 1 <= ci <= 10^6

Código Iterativo

```
int n, x;
cin>>n>>x;
vi ar(n), dp(x + 1, MOD);
dp[0] = 0;
fore(i, 0, n) cin>>ar[i];
fore(i, 1, x + 1)
{
    fore(j, 0, n)
        if(ar[j] <= i)
            dp[i] = min(dp[i], dp[i - ar[j]] + 1);
}
if(dp[x] == MOD)
    cout<<-1<<'\n';
else
    cout<<dp[x]<<'\n';
```

Código Recursivo

```
vi monedas;
bool vis[tam];
int dp[tam];
int f(int dinero) {
    if(dinero < 0) return tam;
    if(dinero == 0) return 0;
    if(vis[dinero]) return dp[dinero];
    vis[dinero] = 1;
    dp[dinero] = tam;
    for(int x : monedas)
        dp[dinero] = min(dp[dinero], 1 + f(dinero - x));
    return dp[dinero];
}
```

Mochila

Hay N ítems numerados de 1, 2, … , n. Cada ítem tiene un peso wi y un valor vi . Tenemos una mochila que aguanta un peso máximo de W. ¿Cuál es la máxima suma de valores que podemos cargar en la mochila sin que la suma de sus pesos sobrepase W?

Probar aquí

- 1 ≤ N ≤ 100
- 1 ≤ W ≤ 10^5
- 1 ≤ wi ≤ W
- 1 ≤ vi ≤ 10^9

Mochila

Al ver un ítem tenemos dos opciones:

- Añadir el ítem a la mochila
- Ignorarlo y revisar el siguiente

A esto le decimos el método “tomar o no tomar”. Con esta idea se resuelven muchos problemas de programación dinámica

$$mochila(i, pesoRestante) = max \begin{cases} mochila(i+1, pesoRestante - w_i) + v_i \\ mochila(i+1, pesoRestante) \end{cases}$$

Mochila

Casos Base:

- $mochila(i, 0) = 0$ cuando ya no queda espacio en la mochila
- $mochila(N, peso) = 0$ cuando hemos visto todos los ítems $(0 .. n - 1)$

Tiempo:

- Hay N ítems y W posibles valores de peso restante, por lo tanto hay $N \times W$ estados
- Cada estado solo tiene dos relaciones, tomar o no tomar, $O(1)$ • La complejidad es $O(NW)$

Código

```
int n, w[101], v[101], dp[101][100001];
int mochila(int i, int peso) {
    if (i == n) return 0;
    if (peso == 0) return 0;
    if (dp[i][peso] != -1) return dp[i][peso];
    int res = 0;
    // tomar
    if (peso >= w[i])
        res = max(res, mochila(i+1, peso - w[i]) + v[i]);
    // no tomar
    res = max(res, mochila(i+1, peso));
    return dp[i][peso] = res;
}
```

Longest Increasing Subsequence (LIS)

La máxima subsecuencia creciente (LIS) de un array es la subsecuencia más larga donde cada elemento es mayor que el anterior.

Probar aquí

Ejemplo:

Dado el array [10, 22, 9, 33, 21, 50, 41, 60, 80], la LIS es [10, 22, 33, 41, 60, 80].

Longest Increasing Subsequence (LIS)

Enfoque Básico:

Utilizar un arreglo dp donde dp[j] es el elemento más pequeño en el que termina una subsecuencia creciente de tamaño j + 1.

Transición (para cada elemento de ar actualizar la tabla):

- j = primer elemento del dp mayor o igual a ar[i]. Si todos son menores, el arreglo de dp se extiende en 1
- dp[j] = mínimo entre dp[j] y ar[i]

Complejidad:

El enfoque básico tiene una complejidad de O(n^2), se puede optimizar con búsqueda binaria para que sea O(nlogn)

Longest Increasing Subsequence (LIS) Código

```
int dp[N], ar[N], n, lis = 0;
for (int i = 0; i < n; ++i) {
  // increasing: lower_bound
  // non-decreasing: upper_bound
  int j = lower_bound(dp, dp + lis, ar[i]) - dp;
  dp[j] = min(dp[j], ar[i]);
  lis = max(lis, j + 1);
}
```

Longest Common Subsequence (LCS)

La subsecuencia común más larga (LCS) es la subsecuencia más larga que es común en dos secuencias.

Probar aquí

Ejemplo:

Dadas las secuencias "AGGTAB" y "GXTXAYB", la LCS es "GTAB".

Longest Common Subsequence (LCS)

Enfoque Básico:

Utilizar una matriz dp donde dp[i][j] representa la longitud de la LCS de las primeras i letras de la primera secuencia y las primeras j letras de la segunda secuencia.

Transición:

dp[i][j] = max(dp[i-1][j], dp[i][j-1])

Si X[i-1] == Y[j-1]:

    entonces dp[i][j] = max(dp[i][j], dp[i-1][j-1] + 1).

Longest Common Subsequence (LCS)

	#	G	X	T	X	A	Y	B
#	0	0	0	0	0	0	0	0
A	0	0	0	0	0	1	1	1
G	0	1	1	1	1	1	1	1
G	0	1	1	1	1	1	1	1
T	0	1	1	2	2	2	2	2
A	0	1	1	2	2	3	3	3
B	0	1	1	2	2	3	3	4

Longest Common Subsequence (LCS) Código

```
int longestCommonSubsequence(string text1, string text2) {
    int n = text1.size(), m = text2.size();
    vector<vector<int>> lcs(n+1, vector<int>(m+1));
    for(int i = 1; i <= n; i++)
        for(int j = 1; j <= m; j++)
        {
            lcs[i][j] = max(lcs[i - 1][j], lcs[i][j - 1]);
            if(text1[i - 1] == text2[j - 1])
                lcs[i][j] = max(lcs[i][j], lcs[i - 1][j - 1] + 1);
        }
    return lcs[n][m];
}
```

Recorrido óptimo en grilla

Dada una grilla de enteros no negativos, determinar el camino desde la esquina superior izquierda a la esquina inferior derecha que maximice la suma de los números en el camino, dado que sólo nos podemos mover hacia abajo o hacia la derecha.

5	2	1	0
2	1	5	3
0	4	4	3
1	5	2	4

(Respuesta: 24)

Recorrido óptimo en grilla

Definimos F(i,j) = “Valor del mayor camino hasta la posición (n, m) si empezamos desde la posición (i, j)”

Por lo tanto, Respuesta = F(1,1)

Caso base: F(n,m) = Grid(n,m)

Casos recursivos:

F(i,m) = Grid(i,m) + F(i+1,m) (i < n)

F(n,j) = Grid(n,j) + F(n,j+1) (j < m)

F(i,j) = Grid(i,j) + max(F(i+1,j) , F(i,j+1)) (i < n && j < m)

Recorrido óptimo en grilla

5	2	1	0
2	1	5	3
0	4	4	3
1	5	2	4

Recorrido óptimo en grilla

5	2	1	0
2	1	5	3
0	4	4	3
1	5	2	4 4

Recorrido óptimo en grilla

5	2	1	0
2	1	5	3
0	4	4	3
1	5	2 6	4 4

Recorrido óptimo en grilla

5	2	1	0
2	1	5	3
0	4	4	3
1	5 11	2 6	4 4

Recorrido óptimo en grilla

5	2	1	0
2	1	5	3
0	4	4	3
1 12	5 11	2 6	4 4

Recorrido óptimo en grilla

5	2	1	0
2	1	5	3
0	4	4	3 7
1 12	5 11	2 6	4 4

Recorrido óptimo en grilla

5	2	1	0
2	1	5	3
0	4	4 11	3 7
1 12	5 11	2 6	4 4

Recorrido óptimo en grilla

5	2	1	0
2	1	5	3
0	4 15	4 11	3 7
1 12	5 11	2 6	4 4

Recorrido óptimo en grilla

5	2	1	0
2	1	5	3
0 15	4 15	4 11	3 7
1 12	5 11	2 6	4 4

Recorrido óptimo en grilla

5	2	1	0
2	1	5	3 10
0 15	4 15	4 11	3 7
1 12	5 11	2 6	4 4

Recorrido óptimo en grilla

5	2	1	0
2	1	5 16	3 10
0 15	4 15	4 11	3 7
1 12	5 11	2 6	4 4

Recorrido óptimo en grilla

5	2	1	0
2	1 17	5 16	3 10
0 15	4 15	4 11	3 7
1 12	5 11	2 6	4 4

Recorrido óptimo en grilla

5	2	1	0
2 19	1 17	5 16	3 10
0 15	4 15	4 11	3 7
1 12	5 11	2 6	4 4

Recorrido óptimo en grilla

5	2	1	0 10
2 19	1 17	5 16	3 10
0 15	4 15	4 11	3 7
1 12	5 11	2 6	4 4

Recorrido óptimo en grilla

5	2	1 17	0 10
2 19	1 17	5 16	3 10
0 15	4 15	4 11	3 7
1 12	5 11	2 6	4 4

Recorrido óptimo en grilla

5	2 19	1 17	0 10
2 19	1 17	5 16	3 10
0 15	4 15	4 11	3 7
1 12	5 11	2 6	4 4

Recorrido óptimo en grilla

5 24	2 19	1 17	0 10
2 19	1 17	5 16	3 10
0 15	4 15	4 11	3 7
1 12	5 11	2 6	4 4

Recorrido óptimo en grilla - Código Iterativo

```
int G[MAX_N][MAX_M];
int n, m ;
int dp[MAX_N][MAX_M];
void calc_dp_iter () {
    for (int i = n - 1; i >= 0; --i) {
        for(int j = m - 1; j >= 0; --j) {
            if(i == n - 1 && j == m - 1) dp[i][j] = G[i][j];
            else {
                dp[i][j] = 0;
                if(i < n - 1)
                    dp[i][j] = max(dp[i][j], G[i][j] + dp[i + 1][j]);
                if(j < m - 1)
                    dp[i][j] = max(dp[i][j], G[i][j] + dp[i][j + 1]);
            }
        }
    }
}
```

Recorrido óptimo en grilla - Código Recursivo

```
int G[MAX_N][MAX_M];
int n, m;
int dp[MAX_N][MAX_M]; // inicializar en -1
int f (int i , int j) {
    int &r = dp[i][j]; // (&) es una referencia para actualizar más fácil
    if(r != -1) return r;
    if(i == n - 1 && j == m - 1) r = G[i][j];
    else {
        r = 0;
        if(i < n - 1) r = max(r, G[i][j] + f(i + 1, j));
        if(j < m - 1) r = max(r, G[i][j] + f(i, j + 1));
    }
    return r;
}
```

Recorrido óptimo en grilla - generar solución

Dado que tenemos guardada la respuesta para cada subproblema, es sencillo saber si un movimiento es óptimo o no:

Es óptimo moverme hacia abajo desde (i, j) cuando:

```
F(i,j) = Grid(i,j) + F(i+1,j)
```

Es óptimo moverme hacia la derecha desde (i, j) cuando:

```
F(i,j) = Grid(i,j) + F(i,j+1)
```

Usando esto, es sencillo generar una solución:

Empiezo en (1, 1) y realizo movimientos óptimos hasta (n, m).

Recorrido óptimo en grilla - generar solución

5 24	2 19	1 17	0 10
2 19	1 17	5 16	3 10
0 15	4 15	4 11	3 7
1 12	5 11	2 6	4 4

Recorrido óptimo en grilla - código de generar solución

```
string sol = "";
// ( gen_sol_from debe ser llamada despues de calcular dp)
void gen_sol_from(int i, int j) {
    if(i == n - 1 && j == m - 1) return ;
    if (i < n - 1 && dp[i][j] == G [i][j] + dp[i + 1][j]) {
        sol += 'A'; // ( Abajo )
        gen_sol_from(i + 1, j);
    } else {
        sol += 'D'; // ( Derecha )
        gen_sol_from(i, j + 1);
    }
}
```

Recorrido óptimo en grilla - Contar soluciones

Se puede contar la cantidad de soluciones una vez que sabemos para cada paso si es óptimo o no, usando otra DP.

Definimos C(i,j) como:

“Cantidad de caminos óptimos desde la posición (i, j) hasta (n, m)”

Caso base:

C(n,m) = 1

Caso recursivo:

C(i, j) = 0

C(i,j) += C(i+1,j) (F(i,j) == Grid(i,j) + F(i+1,j) && i < n)

C(i,j) += C(i,j+1) (F(i,j) == Grid(i,j) + F(i,j+1) && j < m)

Recorrido óptimo en grilla - Código de contar soluciones

```
int cnt[MAX_N][MAX_M];
int calc_num_sols() {
    for (int i = n - 1; i >= 0; --i) {
        for(int j = m - 1; j >= 0; --j) {
            if(i == n - 1 && j == m - 1) cnt[i][j] = 1;
            else {
                cnt[i][j] = 0;
                if(i < n - 1 && dp[i][j] == G[i][j] + dp[i + 1][j])
                cnt[i][j] += cnt[i + 1][j];
                if(j < m - 1 && dp[i][j] == G[i][j] + dp[i][j + 1])
                cnt [i][j] += cnt[i][j + 1];
            }
        }
    }
    return cnt[0][0];
}
```

Recorrido óptimo en grilla - generar k-ésima solución

A veces, no nos piden cualquier solución, sino la k-ésima lexicográfica.

5	2	1	0
2	1	5	3
0	4	4	3
1	5	2	4

En este caso, las soluciones (ordenadas lexicográficamente) son:

1 ADDADA

2 DADADA

3 DDAADA

La solución es similar a encontrar cualquier solución, excepto que decidimos qué paso hacer dependiendo de la cantidad de soluciones de los subproblemas.

Recorrido óptimo en grilla - código generar k-ésima solución

```
string sol = "";
void gen_kth(int i, int j, int k) {
    if(i == n - 1 && j == m - 1) return;
    if(i < n - 1 && dp [i][j] == G[i][j] + dp[i + 1][j]) {
        // si moverme para abajo es optimo
        if(cnt[i + 1][j] >= k) {
            sol += 'A'; // (si hay al menos k soluciones hacia abajo)
            gen_kth(i + 1, j, k);
            return;
        }
        // si ir hacia abajo es optimo, pero la k-esima solucion no cae ahi,
        // descuento la cantidad de soluciones hacia abajo antes de seguir
        k -= cnt [i + 1][j];
    }
    sol += 'D';
    gen_kth(i, j + 1, k);
}
```

Recorrido óptimo en grilla - Optimización de memoria

- Podemos hacer una optimización de memoria, sabiendo que en cada paso sólo accedemos al valor del subproblema en la fila actual o en la siguiente, pero nunca más allá.
- Por lo tanto, si lo hacemos iterativamente, sólo tenemos que guardar la fila actual y la que procesamos anteriormente.
- Esta optimización suele no ser necesaria, pero es importante tenerla en cuenta si la programación dinámica consume mucha memoria, o el problema está ajustado en tiempo.

Recorrido óptimo en grilla - código optimización de memoria

```
int dp[2][MAX_M];
int calc_dp_iter() {
    for (int i = n - 1; i >= 0; --i) {
        for(int j = m - 1; j >= 0; --j) {
            if(i == n - 1 && j == m - 1) dp[i % 2][j] = G[i][j];
            else {
                dp[i % 2][j] = 0;
                if(i < n - 1)
                    dp[i % 2][j] = max(dp[i % 2][j], G[i][j] + dp[1 - i % 2][j]);
                if(j < m - 1)
                    dp[i % 2][j] = max(dp[i % 2][j], G[i][j] + dp[i % 2][j + 1]);
            }
        }
    }
    return dp[0][0];
}
```

Recursos Adicionales

- Clase DP de Argentina: https://www.youtube.com/watch?v=1TCdnXpQw6c&t=8784s
- Errichto en YT: https://youtu.be/YBSt1jYwVfU
- USACO: https://usaco.guide/gold/intro-dp?lang=cpp
- Lista con mucha información: https://codeforces.com/blog/entry/67679